Å Energi



HERSS

The Hydraulic Economic River System Simulator



User manual and technical documentation
Version 4.1.0

Bernt Viggo Matheussen

Nov 8, 2026

MIT License

The HERSS software, test data and the user manual are released under the MIT license:

Copyright © 2026 [Bernt Viggo Matheussen, Å Energi]

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Please look at the following websites to learn more about the MIT license.

- `https://en.wikipedia.org/wiki/MIT_License`
- `https://opensource.org/license/mit`

Preface

Welcome to the user manual and technical documentation for the Hydraulic Economic River System Simulator (HERSS). HERSS is a lightweight and flexible simulation model for analysing the hydrology, hydraulics, and operation of regulated river systems, with particular emphasis on hydropower applications. The model can be used to simulate how water moves through a river system, how reservoirs and power stations interact, and how operational decisions and market conditions influence system behaviour over time.

This document is intended to provide enough information for users to get started with HERSS and to use the software effectively. A basic background in hydrology, river hydraulics, and hydropower is beneficial, but not strictly required. Some familiarity with working in a terminal on Ubuntu/Linux, or in Windows Subsystem for Linux (WSL), is also helpful when running the software and examples.

Regardless of your level of experience, the aim of this manual is to help you get up to speed with setting up simulations, understanding the model input and output, and using HERSS to analyse the dynamics of regulated river systems.

Please note that both the HERSS software and this documentation are still work in progress. The model is under active development, and some parts of the code, test datasets, and documentation may contain errors, inconsistencies, or incomplete descriptions. Not all details of the current HERSS version are yet fully documented, and in some cases the source code remains the most complete reference for how calculations are performed.

HERSS is developed as an open-source project, and contributions are welcome. Users and developers are encouraged to report issues, suggest improvements, and participate in further development of the model. The long-term goal of the HERSS project is to provide a fast, transparent, and adaptable simulation tool for a wide range of analyses of regulated river systems.

The HERSS model has been developed over several years, and many summer interns in Å Energi have used the software. The software has also been used in master thesis work, e.g. at the Univerity of Bergen. The timeline of major contributions in the HERSS project is shown in table 1.

Minor contributions, ideas, bug reports, etc. have been made by Daniel Lindestad, Jonas Riisøen Land, Emilio Lombardo, and Daniel Grønn.

The HERSS project is supported by Å Energi (www.aenergi.no)

Bernt Viggo Matheussen, Nov 8, 2026.

Timeline	Changes/Contributions
Spring 2023	Initial code, testdata and documentation developed by BVM
Summer 2023	Development of testdata for Tovdalen Artifical Hydro Power System (TAHPS), by Kim Ly
Spring 2024	GitHub repository established, by BVM
Summer 2025	Main changes in the project: Added multi-generator support for up to six generators per power station. Added multi-temporal- resolution for simulation of varying timesteps. Added fast overflow. Added date check. Developed testing suite for HERSS functionality. This work was done by Ove Arstad.
Spring 2026	Major upgrade and merging of new functionality (multi-resolution, multi-generator, new routing algorithm, etc). New documentation, plots, new test datasets, quality control, logging, etc. BVM May 2026.
Summer 2026	Terje Sandø implemented pumping, qmin, updated docs, minor bug fixes. New test riversystem.
Fall 2026	BVM implemented three new routing algorithms. Added a new nodetype: Intake. Added various testdatasets, minor bug fixes. Updated doc.
Fall 2026	Andreas Wold Sløgedal provided a fix with pumping issues. Also contibuted with a new testdataset.

Table 1: Timeline of the development and contributions to the HERSS project

Contents

1 Getting started **1**
- 1.1 System Requirements . . . 1
- 1.2 Installation instructions . . . 2
- 1.3 Running HERSS on the commandline . . . 3
- 1.4 Topology File and River System Structure . . . 5
- 1.5 Initial State File . . . 6
- 1.6 Time Series Input Data . . . 7
- 1.7 Output Data from HERSS . . . 8
  - 1.7.1 Overview of the Output Files . . . 9
  - 1.7.2 Aggregated River System Output . . . 9
  - 1.7.3 Reservoir Output . . . 10
  - 1.7.4 Node-Level Output . . . 10
  - 1.7.5 State Output . . . 11
  - 1.7.6 Use of the Output Data . . . 11

2 Model structure **13**
- 2.1 Nodetypes, Topology and ordering . . . 14
  - 2.1.1 Nodetypes . . . 14
  - 2.1.2 Topology and ordering . . . 14
- 2.2 Reservoir . . . 17
- 2.3 Powerstation . . . 24
- 2.4 Channel . . . 30
  - 2.4.1 Routing Model: Cascaded Linear Reservoirs . . . 30
  - 2.4.2 Routing Model: Nonlinear Reaches . . . 31
  - 2.4.3 Parameters . . . 32
  - 2.4.4 Minimum Flow (`QMIN`) . . . 33
  - 2.4.5 State File . . . 34
  - 2.4.6 Channel Start and End State Files . . . 35
- 2.5 Pump . . . 37
  - 2.5.1 Hydraulic Head and Power Calculation . . . 37
  - 2.5.2 Topology File Syntax . . . 38

2.5.3 Start and Stop Costs . . . 38
2.5.4 Numbering Requirement . . . 39
2.6 Intake . . . 40
2.7 Automatic Differentiation (AD) . . . 41
2.7.1 Value function and gradients . . . 41
2.7.2 How the adjoint is evaluated . . . 41
2.7.3 Enabling gradient output . . . 42
2.7.4 Optimizing actions with Adam . . . 42
2.7.5 Example datasets . . . 43
2.7.6 Current limitations . . . 43
2.8 Input . . . 44
2.8.1 The Global Configuration File . . . 44
2.8.2 The Topology File . . . 45
2.8.3 The Price file . . . 46
2.8.4 The Inflow file . . . 47
2.8.5 The Actions File . . . 48
2.8.6 The State File . . . 49
2.9 Output Files . . . 51
2.9.1 Node Output Files . . . 51
2.9.2 Reservoir Filling Fractions File . . . 52
2.9.3 River System Summary File . . . 52
2.10 Multi-Temporal Resolution and date formats . . . 53

3 Tutorial and feature guide **55**
3.1 Testdata Reservoir Cascade A . . . 55
3.2 Testdata UTAHPS Intake . . . 59
3.3 Running HERSS from Python . . . 59
3.3.1 What is cppyy? . . . 59
3.3.2 Loading HERSS into Python . . . 60
3.3.3 Initialising the Logger . . . 60
3.3.4 Setting Up a Simulation . . . 60
3.3.5 Reading and Writing Simulation State . . . 61
3.3.6 The pyherss.py Example Script . . . 62

4 Datasets **63**
4.1 Upper Tovdalen Artificial Hydro Power System (uTAHPS) . . . 63
4.1.1 Riversystem . . . 63
4.1.2 Timeseries . . . 66

5 Topology Editor **69**
5.1 Installing the Python Environment . . . 69
5.2 Starting the Editor . . . 70
5.3 Viewing and Editing a Topology . . . 70
5.4 Displaying Simulation Results . . . 71

6 Known Issues and Future Work **74**
6.1 Known Issues . . . 74
6.1.1 Automatic Minimum Flow Release . . . 74
6.1.2 Minimum Flow Constraints in Channels (QMIN) . . . 74
6.1.3 Maximum Adjustment Penalty . . . 74
6.1.4 Generator Max Discharge . . . 74
6.1.5 Start of Multi-Generator . . . 75
6.2 Future Work . . . 75
6.2.1 Multi-Reservoir Water Value Calculation . . . 75
6.2.2 Flood Level Penalty . . . 75
6.2.3 Interface to python . . . 75
6.2.4 Variable Timestep Support Throughout . . . 75
6.2.5 Parallel Simulation . . . 75
6.2.6 Improved Topology File Validation . . . 76
6.2.7 Output and Visualisation . . . 76
6.2.8 Automatic generation of node networks . . . 76

List of Tables

1 Timeline of the development and contributions to the HERSS project iv

2.1 Keywords in the global configuration file. 44
2.2 Example of variable temporal resolution input. The time step Δt is derived internally from the difference between consecutive timestamps. 53

3.1 Commonly used `Herss` interface methods available from Python. . . 62

List of Figures

1.1 Screenshot of terminal window after running the `g++` command. . . 2
1.2 Screenshot of terminal window after running the `make` command. . . 3
1.3 Screenshot of terminal window after running HERSS. 4
1.4 Screenshot of terminal/console showing the logfile. 4
1.5 Node structure and river topology of the mini uTAHPS system . . . 5
1.6 Input timeseries (inflow, price, and actions) for the mini uTAHPS . 7
1.7 Output timeseries for the mini uTAHPS 8

2.1 Schematic of a hydropower river system represented as a network of nodes in HERSS. 13
2.2 Node type reservoir with the four outlet options. 18
2.3 Reservoir with parametric geometry 19
2.4 Node type powerstation. 24
2.5 Cascaded linear reservoirs 31
2.6 Node type `PUMP`. Illustration of the connection of the pump between its source and target reservoirs. 37
2.7 Schematic of the `Mini_pump_test` system, and how the nodes are connected in the topology. `IDNR` 0 is the pump, and `IDNR` 3 is the power station. This illustrates how a pump can move water without following the normal rule of water flow from a node with a lower to a higher `IDNR`, but may move water between tow nodes with `IDNR` higher than itself. 39
2.8 Schematic of node type Intake 40

3.1 Node structure and topology of the reservoir cascade test dataset . 55
3.2 Simulation results for the reservoir RES_A (node 0) 57
3.3 Simulation results for the powerstation PSTAT_B (node 4), and the connected reservoir RES_B (node 4) 58
3.4 Intake nodestructure 59

4.1 Map of Upper Tovdalen Artificial Hydro Power System 64
4.2 Topology of the uTAHPS. Node number and names included. . . . 65

4.3 Timeseries plot of the inflow to four reservoirs in uTAHPS 66
4.4 Multi-temporal timeseries plot of electricity price 68

5.1 An example HERSS dataset displayed in the Topology Editor. Nodes represent system components, and links show their connections. . . 72
5.2 A second HERSS dataset shown in the Topology Editor. The diagram can be rearranged without changing simulation parameters or node IDs. 73

Chapter 1

Getting started

This chapter helps you to get started. Make sure you have downloaded or cloned the github repository (`https://github.com/berntmath/herss`).

1.1 System Requirements

The HERSS model is written in C++. It has been developed and tested using `g++` and Make, running on a Ubuntu operating system (20.04). It has also been tested on Ubuntu installed under the Windows Subsystem for Linux (WSL) and on standalone servers. The HERSS have successfully been compiled and run using `g++` version 9.3.0 and 9.4.0, and the GNU Make 4.2.1 program. The software has also been tested on Ubuntu 22.04 using `g++` 11.4.0. It compiled and ran without errors. The HERSS uses only standard C++ libraries and has no other dependencies. It should compile and run on most computers having a C++ compiler. The hardware requirements for the HERSS depend on how you intend to use the model. The test datasets that are part of the code has relatively low memory and CPU requirements and should run on almost any computer.

1.2 Installation instructions

Download files from GitHub and unpack in a project directory. You should have a directory structure as shown in the figure below:

```
herss/
├── src/
├── src_tests/
├── py_src/
├── doc/
├── data/
│   ├── mini_utahps_daily/
│   ├── mini_utahps_hourly/
│   ├── res_casc_A/
│   ├── utahps_daily/
│   ├── utahps_hourly/
│   └── utahps_multires/
├── README.md
└── LICENSE.md
```

Open a terminal window and navigate into the `src` directory. Run `g++ --version` on the command line. You should now see the version number and some additional text (similar to Figure 1.1). If it fails, you need to install `g++` on your system.

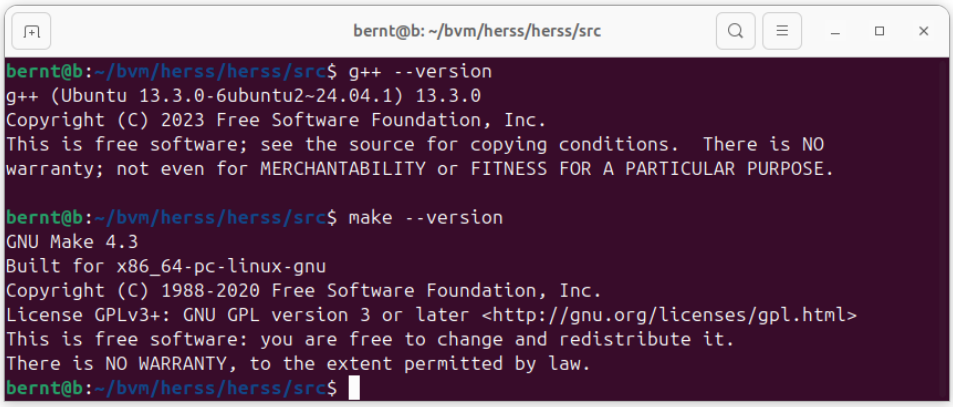


Figure 1.1: Screenshot of terminal window after running the `g++` command.

Now type in `make --version` on the command line. You should now see the version number and some additional text. Your terminal window should look similar to Figure 1.1.

Open a terminal window and navigate into the `src` directory. Run `make` on the command line. You should now se something similar to what is shown in Figure 1.2. Look also for the files (*herss.exe* and *herss.so*). We need them later when we run the HERSS model from commandline or from within python.

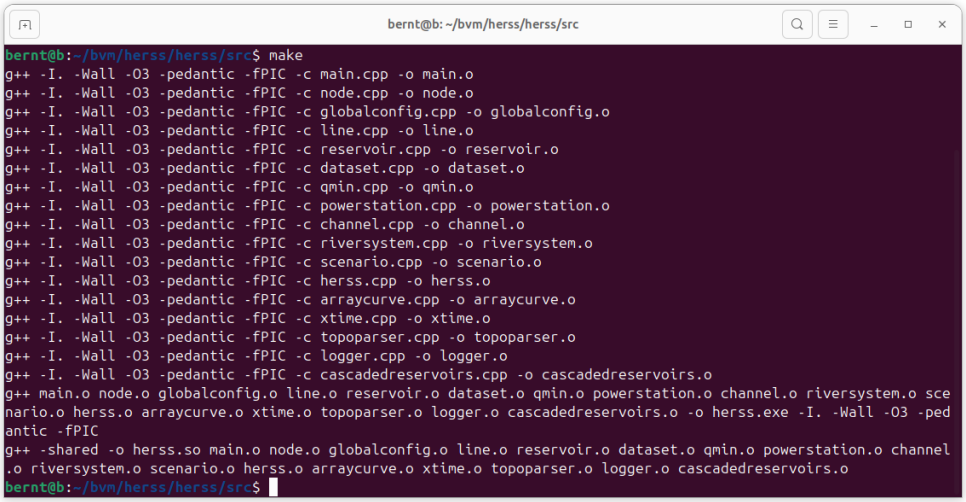


Figure 1.2: Screenshot of terminal window after running the `make` command.

1.3 Running HERSS on the commandline

After you have downloaded and compiled the HERSS model you are now ready to use it. The HERSS model has no graphical user interface. It is run in a terminal/command window. We will now try to run a mini version of a riversystem (three nodes) using one of the datasets for the Upper Tovdalen Artificial Hydro Power System (uTAHPS). Navigate to the folder: `./mini_utahps_hourly/`. You should see the folder `./output/` and the following files:

- *actions.txt*
- *global.txt*
- *inflowseries.txt*
- *pricefile.txt*

- *start_state.txt*
- *topology.txt*

Throughout this chapter, we will use the test dataset located in *mini_uTAHPS_hourly*. Now run the HERSS model with the command `../../src/herss.exe global.txt`.

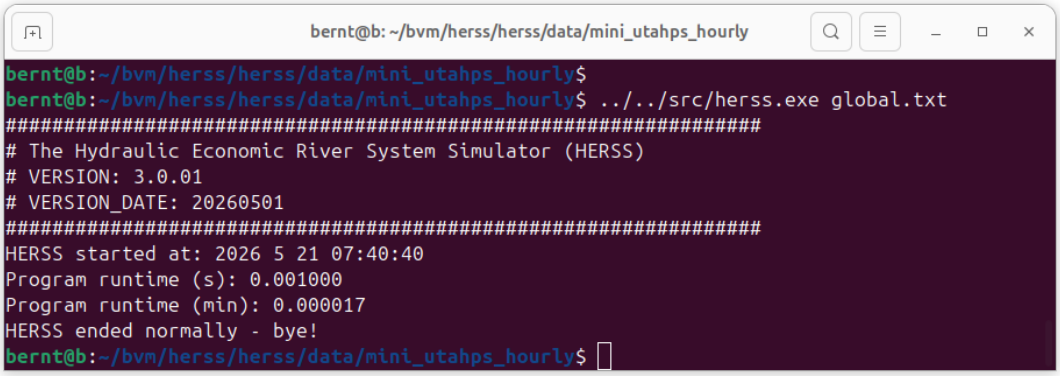


Figure 1.3: Screenshot of terminal window after running HERSS.

Congratulations! You have now completed your first simulation with the HERSS model. When HERSS started, it first read the global configuration file (*global.txt*). This file tells the model which input files to use, where the input and output folders are located, and which optional settings should be enabled during the run. After that, HERSS read the main input files needed for the simulation. These files includes the topology of the river system, the inflow data, the price data, the action data, and the initial state file.

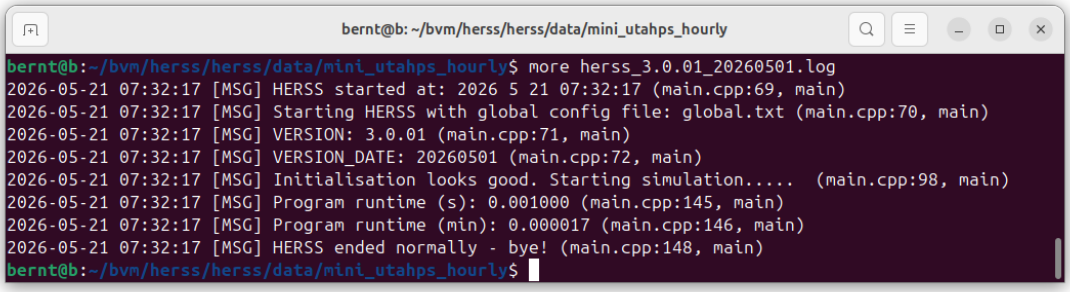


Figure 1.4: Screenshot of terminal/console showing the logfile.

At the end of the simulation, HERSS wrote the result files to the output folder and also created a log file: *herss_3.0.01_20260501.log* The log file contains

useful information about the model run, including startup messages, version information, progress messages, warnings, and possible error messages. It can be helpful when checking that the simulation completed as expected or when diagnosing problems in the input files or model setup.

If you use the command `more herss_3.0.01_20260501.log`, you will see something similar to Figure 1.4.

1.4 Topology File and River System Structure

The river system in this example is defined in the topology file *topology.txt*. The purpose of the topology file is to describe the physical structure of the hydropower system and to provide the technical data needed for each node in the model. In HERSS, a river system is built up from a sequence of nodes, where each node represents a physical component such as a reservoir, a power station, or a river reach/channel.

Figure 1.5 shows a schematic of the mini system used in this example. The same structure is described in the topology file (*topology.txt*). We have one reservoir named `Hjelle`. This is connected to the powerstation `Svoletjonn`. If the water level in the reservoir is above the highest regulated water height (HRW), the water will flow into the channel called `Vanarosen`. In the powerstation `Svoletjonn`, the outflow from the station goes into the `Vanarosen` channel.

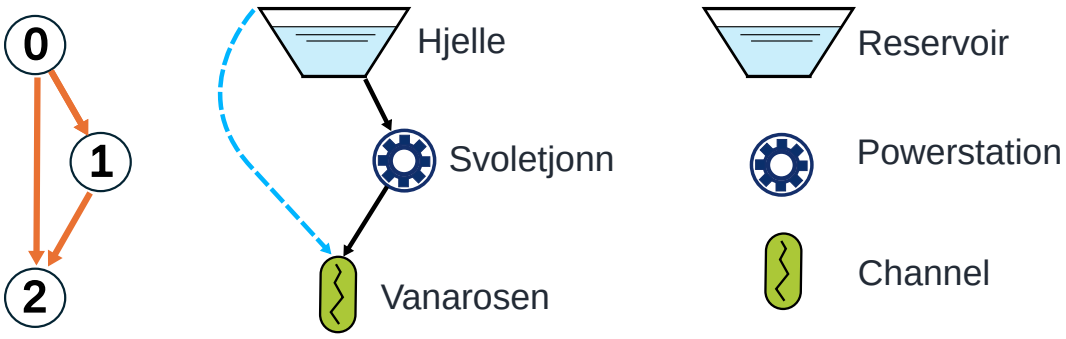


Figure 1.5: Node structure and river topology of the mini uTAHPS system

The topology file is divided into node blocks. Each block starts with a `NODE` line and ends with `ENDNODE`. The `NODE` line gives the node type, the node ID, and the node name. In this way, the file defines both the components of the river system and how they are connected.

HERSS currently uses three main node types:

- `RESERVOIR`: a storage node where water is accumulated and released.

- `PSTATION`: a power station node where water is converted into hydropower production.
- `CHANNEL`: a river reach or routing node that transports water downstream.

Each node block contains the parameters that are relevant for that node type. For example, a reservoir node contain water levels, storage curves, and outlet definitions, while a power station node contains generator and efficiency data. A channel describes the hydraulic routing with traveltime, etc.

1.5 Initial State File

The initial state file defines the hydraulic and operational state of the river system at the start of the simulation. In other words, it tells HERSS how much water is stored in reservoirs and channels, and what the initial condition of other dynamic components should be before the first time step is simulated.

The initial state is given in the file *start_state.txt*. This file is read during model initialization and provides the starting values needed to begin the simulation from a physically meaningful condition. The initial state file is organized by node type. Each relevant node is listed on a separate line together with the values needed to define its starting condition. For reservoirs, the file gives the initial filling as a fraction of the active storage range. In the mini uTAHPS example, the line

```
NODE RESERVOIR 0 HJELLE 0.67
```

means that reservoir 0, `HJELLE`, starts at a filling fraction of 0.67. This corresponds to 67% of the active storage range between the lower regulated water level and the higher regulated water level. For power stations, the file gives the initial production state. In the example,

```
NODE PSTATION 1 SVOLETJONN 0.0
```

means that the power station `SVOLETJONN` starts with zero production. For channels, the file gives the initial storage in each linear reservoir used by the routing model. In the example,

```
NODE CHANNEL 2 VANAROSEN 0.001 0.002 0.003
```

means that channel 2, `VANAROSEN`, starts with three internal storage values, one for each linear reservoir in the channel cascade. The number of storage values must match the number of cascaded linear reservoirs defined in the topology file.

1.6 Time Series Input Data

In addition to topology and start state, HERSS needs time series input data to run the simulation. In the mini uTAHPS example, the time-dependent input is given in three files: *inflowseries.txt*, *pricefile.txt*, *actions.txt*.

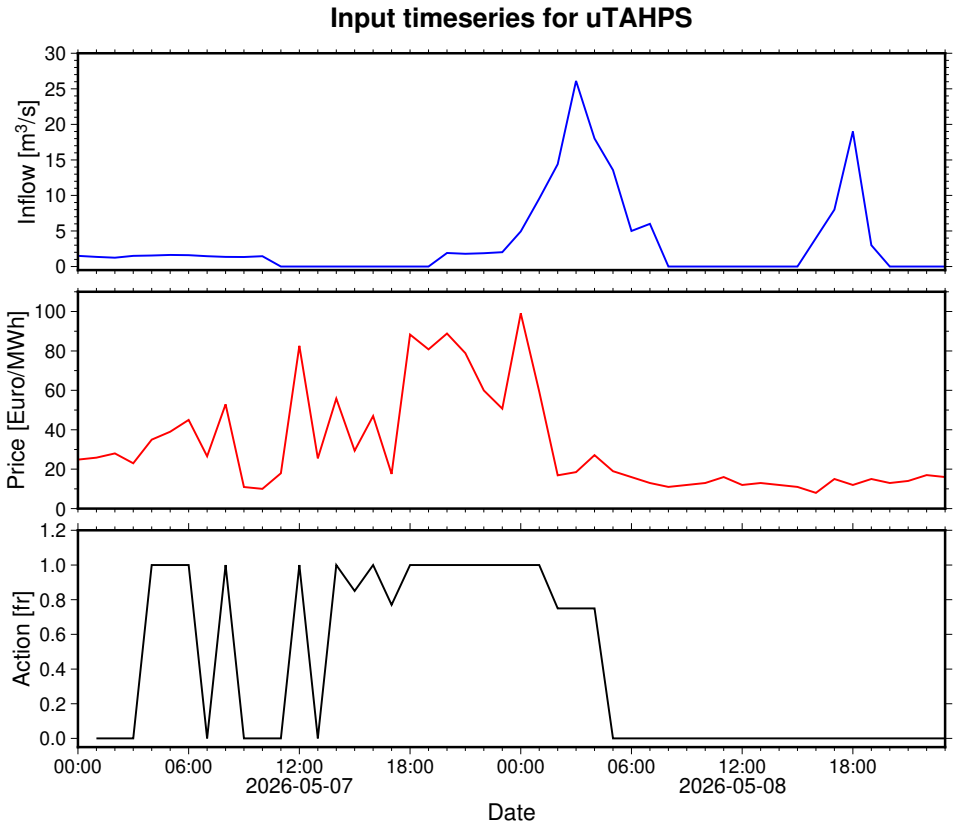


Figure 1.6: Input timeseries (inflow, price, and actions) for the mini uTAHPS

Together, these files describe how much water enters the system, what the electricity price is at each time step, and how the power station is operated during the simulation period. The action variable is a control signal with values between 0.0 and 1.0. It determines the flow through the power station as a fraction of the maximum flow capacity. A value of 0.0 means that the station is off, a value of 1.0 means full operation, and intermediate values represent partial operation. Figure 1.6 shows a timeseries plot of the input testdata.

1.7 Output Data from HERSS

After a simulation has finished, HERSS writes a set of output files to the output folder. These files contain both aggregated results for the full river system and more detailed results for individual nodes such as reservoirs, power stations, and channels. Together, the output files make it possible to inspect the hydraulic behaviour of the system, evaluate operational performance, and analyse the economic results of the simulation.

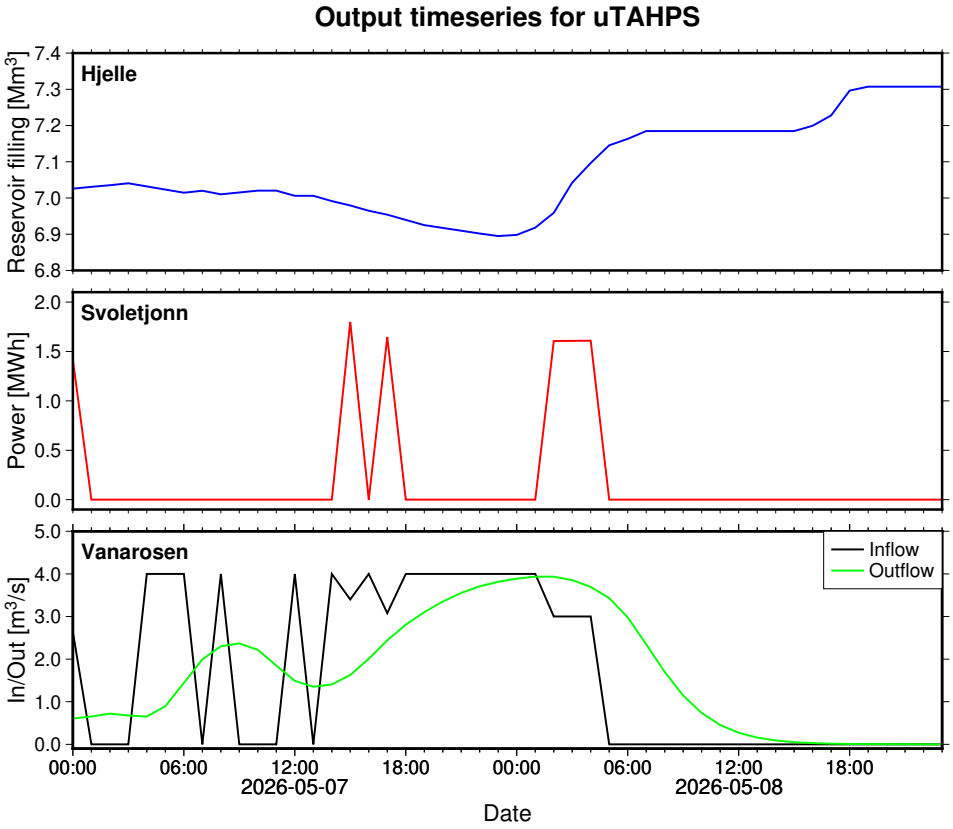


Figure 1.7: Output timeseries for the mini uTAHPS

In practice, the output data can be used to make time series plots of selected variables, for example reservoir levels, hydropower production, and outflow from channels. These plots are often the most convenient way to understand how the simulated river system behaves over time. Figure 1.7 illustrates the reservoir levels, power production and channel flow for the uTAHPS. Note how the reservoir level changes as a function of inflow (see Fig. 1.6) and outflow (production, MWh).

The downstream channel (node 2) in the mini uTAHPS system represents routing of water from the power station further downstream. The channel node

also receives overflow water from the reservoir (Hjelle). In the simulation, the channel does not simply pass inflow directly to the outlet at the same time step. Instead, the flow is delayed and smoothed as it moves through the channel. In HERSS, the channel routing is represented by a cascade of linear reservoirs. This gives a simple but effective description of how water is temporarily stored and released as it travels downstream. For the mini uTAHPS example, the channel is configured with three cascaded linear reservoirs and a travel time parameter of four hours. Together, these settings control how strong the delay and attenuation become.

This behaviour can be seen in the time series plot (Fig. 1.7)) of channel inflow and channel outflow. The inflow curve shows the water entering the channel from upstream, while the outflow curve shows the water leaving the channel after routing through the channel model. In general, the outflow does not respond instantaneously to changes in inflow. Instead, peaks in inflow appear later in time at the outlet. This is the routing delay.

At the same time, the outflow signal is usually smoother than the inflow signal. Sharp peaks in inflow tend to become lower and broader in the outflow series. This effect is known as attenuation. Physically, it reflects temporary storage and redistribution of water inside the channel reach. More on this in chapter 2

1.7.1 Overview of the Output Files

The most important output files are written to the output folder after each model run. Typical files include:

- *riversystem_<systemname>_output.txt*
- *reservoirs_<systemname>_out.txt*
- *outstate.txt*
- optional node output files such as *node0_HJELLE.txt*, *node1_SVOLETJONN.txt*, and *node2_VANAROSEN.txt*

The exact names depend on the system name given in the global configuration file and on whether node-level output has been enabled.

1.7.2 Aggregated River System Output

The file *riversystem_<systemname>_output.txt* contains summary information for the entire river system. This file includes information about the nodes in the

system, the remaining water in the different parts of the river network, and the global water balance.

In addition, this file contains important aggregated economic quantities, such as total income, total cost, total profit, the estimated value of remaining water, and the final value function. These results provide a compact summary of the overall performance of the simulated system over the full time horizon.

1.7.3 Reservoir Output

The file *reservoirs_<systemname>_out.txt* contains time series of reservoir filling for all reservoir nodes in the system. The values are written as reservoir fractions, typically denoted by `fr`, for each simulation time step. This file is particularly useful for plotting reservoir levels over time and for checking whether the reservoirs are being drawn down or refilled as expected during the simulation period.

1.7.4 Node-Level Output

If node-level output is enabled, HERSS writes one file for each node in the system. These files contain the most detailed simulation results and are often the most useful outputs for plotting and interpretation.

For a reservoir node, the output file typically contains variables such as:

- inflow,
- upstream inflow,
- reservoir storage,
- reservoir water level,
- reservoir filling fraction,
- tunnel flow,
- hatch flow,
- overflow, and
- total outflow.

For a power station node, the output file typically contains variables such as:

- upstream inflow,

- action values,
- total outflow,
- head,
- power production,
- income,
- operating cost, and
- profit.

For a channel node, the output file typically contains variables such as:

- upstream inflow,
- channel storage,
- total outflow, and
- possible minimum-flow related costs.

These node-level files are well suited for making time series plots of selected variables, for example reservoir levels, hydropower production, or outflow from a downstream channel.

1.7.5 State Output

HERSS also writes an output state file, typically named *outstate.txt*. This file contains the simulated end state of the system after the last time step. It can be used as a restart state for a later simulation, making it possible to continue from the final condition of a previous model run.

1.7.6 Use of the Output Data

The output data from HERSS can be used for several purposes:

- checking that the simulation behaved as expected,
- verifying water balances and system routing,
- plotting time series of key variables,

- comparing different operational strategies, and
- evaluating the hydraulic and economic performance of the system.
- long term planning analysis
- design of new systems
- optimization of daily power production and reservoir management

For the simple uTAHPS example, useful plots include reservoir level in the Hjelle reservoir, production from the power station, and outflow from the downstream channel. Such plots provide a clear and intuitive way to interpret the model results.

That's it!. Now you have a basic understanding of the HERSS model, how to run it, and what results it produces.

The rest of this document will focus on three parts: model structure, more tutorials and testdata.

Chapter 2

Model structure

HERSS represents a regulated river system as a network of nodes, where each node corresponds to a physical component such as a reservoir, a power station, or a river channel. Every node has a unique identifier and belongs to one of the supported node types. Water enters a node from upstream, is processed according to the physical behavior of that node type, and is then routed to one or more downstream nodes, stepping forward in time until the end of the simulation period.

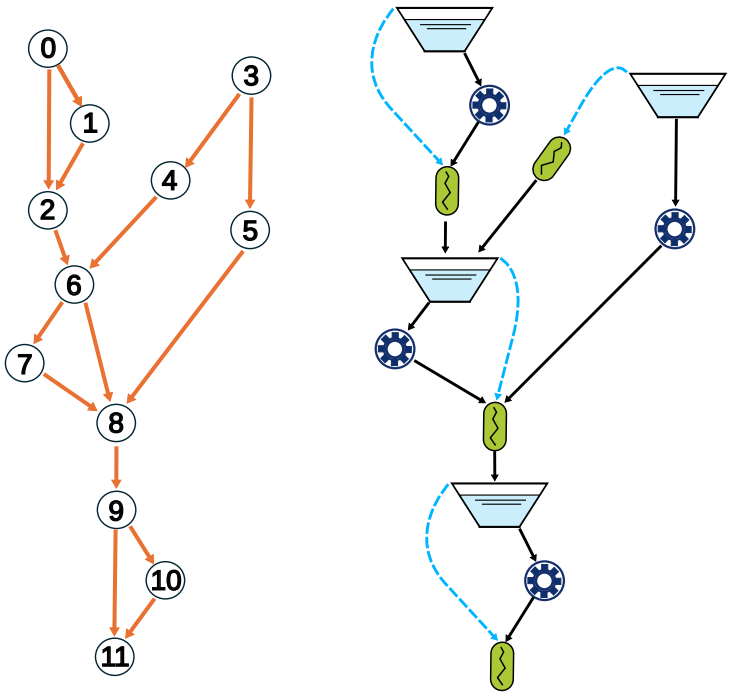


Figure 2.1: Schematic of a hydropower river system represented as a network of nodes in HERSS.

Figure 2.1 shows a schematic of a more complex river network, illustrating how different node types can be combined to represented a realistic hydropower system. This node-based approach makes it straightforward to represent river systems of varying complexity, from a simple single-reservoir system to large networks with many reservoirs, power stations, and channels arranged in branched or cascaded configurations. This chapter describes the supported node types, how they are connected, and what physical processes each one represents.

2.1 Nodetypes, Topology and ordering

2.1.1 Nodetypes

HERSS currently supports four node types:

- **Reservoir**: a storage node representing a regulated lake or impoundment. The reservoir receives inflow, stores water, and releases it through one or more outlets to downstream nodes.
- **Power station**: a node representing a hydropower plant. The power station receives water from an upstream reservoir, generates electricity from the turbine discharge, and passes the discharged water downstream.
- **Channel**: a routing node representing a river reach or transport element. The channel receives water from upstream and routes it downstream with delay and attenuation.
- **Pump**: a node representing a pumped-storage hydropower (PSH) facility, using electricity to move water from a source reservoir to a target reservoir.
- **Intake**: a node representing an intake that captures water from a natural stream. The captured water is passed into a downstream reservoir or channel.

Each node type is described in detail in later sections.

2.1.2 Topology and ordering

The order in which nodes are defined in the topology file determines the order in which they are simulated at each time step. Water is routed from the most upstream node towards the most downstream node in sequence. It is therefore important that the topology file correctly reflects the physical layout and connectivity of the river system.

The topology file defines the river system as a directed acyclic graph (DAG) of nodes, where each node represents either a reservoir, a power station, a channel reach, or a pump (Section 2.5). Each node is assigned a unique integer identifier (`IDNR`), starting at zero. The calculation order in HERSS is determined entirely by the node numbering: at each time step, nodes are simulated in ascending order of their `IDNR`, from node 0 to node $N - 1$, where N is the total number of nodes.

This sequential ordering imposes a strict requirement on how nodes are numbered in the topology file: **all upstream nodes must be assigned lower identifiers than the downstream nodes they drain into, with only one exception for pumped hydro**. In other words, water can only flow from a node with a lower `IDNR` to a node with a higher `IDNR`. When a node is simulated at time step t, it reads the upstream inflow (`up_inflow`) that has already been written by all upstream nodes in the same time step. It then computes its own outflow and adds it to the `up_inflow` accumulator of its downstream node, which has not yet been simulated in that time step.

A concrete consequence is illustrated below. Consider a cascaded system with three nodes:

- Node 0: upstream reservoir
- Node 1: power station, receives discharge from node 0
- Node 2: downstream channel or reservoir, receives discharge from node 1

At each time step, the simulation loop processes node 0, node 1, and node 2 in that order. Node 0 computes its outflow and writes it to `node[1].up_inflow[t]`. Node 1 reads this value, computes its own turbine discharge, and writes it to `node[2].up_inflow[t]`. Node 2 then uses the accumulated inflow for its own computation.

If a node receives water from multiple upstream sources (e.g. a confluence/junction), all contributing upstream nodes must have lower identifiers than the receiving node. Branching topologies, such as a reservoir with both a tunnel outlet and an overflow outlet leading to different downstream nodes, are supported, but all downstream nodes must again have higher identifiers than the source node.

A `PUMP` node, representing a Pumped Storage Hydropower (PSH) facility (Section 2.5) is the one exception to the sequential ordering rule: since it moves water against the normal flow direction, both `PUMP_SOURCE_IDNR` and `PUMP_TARGET_IDNR` must be *higher* than the pump's own `IDNR`.

Numbering Guidelines

The following guidelines should be followed when constructing a topology file:

1. Start numbering at zero (`IDNR = 0`) for the most upstream node in the system.

2. Assign identifiers in strictly increasing order in the downstream direction.

3. At confluences, ensure that *all* contributing nodes have identifiers lower than the receiving node.

4. When placing a `PUMP` node, plan it ahead so it is given a lower `IDNR` than both its source and target reservoirs.

5. The last node in the topology (highest `IDNR`) is typically the most downstream node, for example the ocean or a boundary channel with no downstream connection (indicated by a downstream identifier of `-9` in the topology file).

HERSS does *not* automatically sort or validate the topological order of nodes at runtime. It is the responsibility of the model user to ensure that node identifiers reflect a valid upstream-to-downstream ordering. An incorrect ordering will lead to erroneous water routing, because a downstream node may be simulated before all its upstream contributions have been computed for the current time step, resulting in those contributions being applied one time step too late.

2.2 Reservoir

The node type `RESERVOIR` models a hydro power reservoir. For each reservoir the user must supply a piecewise-linear *reservoir curve* that maps water surface elevation (meters above sea level, masl) to stored volume (Mm^3). A minimum of two points is required, but minimum four points are recommended. The curve must span from below the Lowest Regulated Water (LRW) level to above the Highest Regulated Water (HRW) level, and ideally extend somewhat beyond both limits to avoid extrapolation errors during extreme events.

A reservoir may have up to four independent outlets, each sending water to a specified downstream node identified by its `IDNR`. All outlets are optional except overflow, which is always active. Within each time step, outlets are evaluated in the following fixed order: tunnel, hatch, automatic minimum-flow release, and finally overflow. A schematic of the outlet types is shown in Figure 2.2.

Make a note on modelling LAKES. We can do this by using the Nodetype Reservoir by only using the overflow curve

Tunnel (`OUTLET_TUNNEL`) The primary production outlet, connecting the reservoir to a downstream power station node. The reservoir passes its current water surface elevation to the power station so that the net hydraulic head can be computed. The discharge through the tunnel is determined by the power station's action variable, not by the reservoir directly.

Hatch (`OUTLET_HATCH`) A controllable gate, typically releasing water to a downstream channel. The discharge is a linear interpolation between a minimum flow $Q_{\min}$ and a maximum flow $Q_{\max}$, controlled by the action variable $a \in [0, 1]$:

$$Q_{\text{hatch}} = Q_{\min} + a\,(Q_{\max} - Q_{\min})$$

The hatch only operates when the reservoir level exceeds the sill elevation (`hatch_masl`). An action time series must be supplied for nodes with an active hatch outlet.

Automatic minimum-flow release (`OUTLET_AUTO_QMIN`) A seasonally scheduled release that operates independently of the optimisation action. It is used to enforce regulatory minimum-flow requirements directly from the reservoir, for example a fixed environmental flow obligation. Up to five seasonal periods may be defined, each with its own minimum discharge.

When `OUTLET_AUTO_QMIN` is used, HERSS expect the following arguments:

```
OUTLET_AUTO_QMIN <nr_periods> <downstream_idnr> <outlet_masl>
```

```
<start date DD.MM> <end date DD.MM> <Qmin m3/s>
...
```

where `nr_periods` is the integer describing the number of defined periods listed below (Maximum number of 5). Set `nr_periods` to a non-positive value (e.g. `-9999`) to disable `OUTLET_AUTO_QMIN` entirely. At most one of the defined periods may cross the turn of the year (e.g. `01.10 31.03`). HERSS will return warnings for days in the year which is not covered in the defined periods. Overlapping periods will be ignored, and only the overlapping period with the first start date will be used. The minimum-flow release hatch is placed at `outlet_masl`, and water below this level will not be available for release.

Overflow (`OVERFLOW_CURVE`) Uncontrolled spillage over the dam crest, described by a piecewise-linear curve mapping reservoir elevation to overflow discharge [m^3/s]. Overflow is computed last, after all controlled outlets have been evaluated, and drains to a specified downstream node. Every reservoir must have an overflow curve defined.

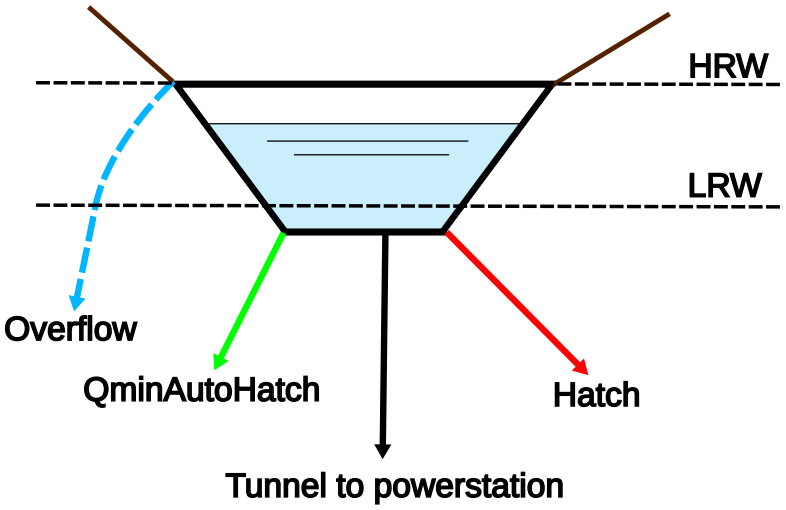


Figure 2.2: Node type reservoir with the four outlet options.

Reservoir geometry

As an alternative to specifying a measured volume–elevation curve (`RESERVOIR_CURVE`), the reservoir shape can be described by a simple parametric geometry (`RESERVOIR_GEOMETRY`) . This option is activated by supplying the five keywords `RESERVOIR_GEOMETRY`, `WIDTH_M`, `LENGTH_M`, `THETA`, and `BOTTOM_MASL` in the reservoir node block. The two representations are mutually exclusive; using both in the same node will cause a model error.

Geometric model The reservoir is approximated as a rectangular basin with uniform cross-section along its length. The side slopes are parameterised by the angle θ, measured from the horizontal. A derived slope term is computed once at initialisation:

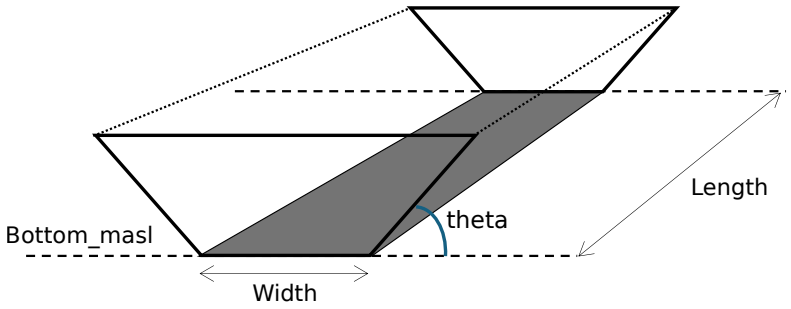


Figure 2.3: Reservoir with parametric geometry

$$s = \tan\left(\frac{(90-\theta)\,\pi}{180}\right) \tag{2.1}$$

The stored volume at water surface elevation z [m.a.s.l.] is then:

$$V(z) = \frac{h \cdot (s \cdot h + W) \cdot L}{10^6} \quad [\mathrm{Mm}^3] \tag{2.2}$$

where $h = z - z_{\mathrm{bottom}}$ is the depth above the reservoir floor and W, L are the bottom width and length respectively. The inverse relation, used to convert a stored volume back to a water surface elevation, is:

$$z = z_{\mathrm{bottom}} + \frac{-L \cdot W + \sqrt{(L \cdot W)^2 + 4 \cdot\ L \cdot s \cdot V}}{2 \cdot L \cdot s} \tag{2.3}$$

Physical interpretation of θ The angle θ describes the inclination of the valley sides:

- $\theta \to 90$: nearly vertical side walls, approximately rectangular cross-section ($s \to 0$).
- $\theta \to 0$: very gently sloping valley ($s \to \infty$, large effective width per unit depth).

A value of $\theta = 20$ represents a moderately V-shaped valley.

Topology file syntax

```
RESERVOIR_GEOMETRY
WIDTH_M     1000
LENGTH_M    1000
THETA       20
BOTTOM_MASL  745.0
```

The parameters are:

Keyword	Type	Description
WIDTH_M	float	Bottom width of the reservoir [m]
LENGTH_M	float	Length of the reservoir [m]
THETA	float	Side-slope angle from horizontal [degrees], $0 < \theta < 90$
BOTTOM_MASL	float	Elevation of the reservoir floor [m.a.s.l.]

Constraints and notes

- RESERVOIR_GEOMETRY and RESERVOIR_CURVE are mutually exclusive; specifying both will cause a model error.
- WIDTH_M and LENGTH_M must be positive; THETA must satisfy $0 < \theta < 90$.
- BOTTOM_MASL must be non-negative and physically plausible.
- The geometry-based volume–elevation relationship is linear, which is a simplification. For reservoirs with strongly irregular bathymetry a measured RESERVOIR_CURVE is more accurate.

Fast Overflow

By default, overflow is computed by interpolating the overflow curve at the current reservoir elevation. For large time steps, or when simulation speed is critical, the FAST_OVERFLOW option can be enabled. With this setting, overflow is calculated simply as the volume above HRW, without interpolating the overflow curve:

$$Q_{\text{overflow}} = \max(V - V_{\text{HRW}},\ 0)$$

This avoids the curve look-up and is numerically more stable for coarse time steps. To enable it, set `FAST_OVERFLOW TRUE` for the relevant reservoir in the topology file.

Spillway

A spillway is a fixed-crest overflow structure that releases water from a reservoir once the water surface elevation exceeds the crest level. It is one of two mutually exclusive methods available for modelling uncontrolled overflow; the other is the `OVERFLOW_CURVE`. Both methods cannot be active simultaneously for the same reservoir node.

Discharge formula The spillway discharge is computed using the standard broad-crested weir formula:

$$Q = C \cdot L \cdot H^{3/2} \tag{2.4}$$

where:

- Q is the spillway discharge [m^3/s],
- C is the dimensionless discharge coefficient [–],
- L is the crest length [m],
- $H = z_{\text{res}} - z_{\text{crest}}$ is the head above the spillway crest [m], computed as the difference between the current reservoir water surface elevation z_{res} and the fixed crest elevation z_{crest}, both in metres above sea level (m.a.s.l.).

Discharge is zero whenever $z_{\text{res}} \leq z_{\text{crest}}$.

Volume limitation The computed discharge volume per time step is capped to prevent the reservoir from draining below the spillway crest level:

$$\Delta V_{\text{overflow}} = \min(Q \cdot \Delta t,\ V_{\text{res}} - V_{\text{crest}}) \tag{2.5}$$

where V_{res} is the current stored volume and V_{crest} is the reservoir volume corresponding to z_{crest}. This ensures numerical stability with large time steps.

Topology file syntax The spillway is configured in the topology file with a single line inside the reservoir node block:

```
SPILLWAY  downstream_idnr  C  L  spillway_level_masl
```

The parameters are:

Field	Type	Description
`downstream_idnr`	integer	Node ID receiving the spillway discharge
`C`	float	Discharge coefficient
`L`	float	Crest length [m]
`spillway_level_masl`	float	Crest elevation [m.a.s.l.]

The downstream node must not be the same as the reservoir itself. The crest elevation z_{crest} may be set below the high reservoir water level (HRW), in which case spillway discharge can occur while the reservoir is still below its design maximum.

Constraints and notes

- `SPILLWAY` and `OVERFLOW_CURVE` are mutually exclusive for a given reservoir; specifying both will cause a model error.
- The discharge coefficient C must be in the range (0, 100] and the crest length L must be positive.
- The crest elevation must be a physically plausible value (positive and below the summit of Mount Everest, used internally as an upper sanity bound).
- If the `FAST_OVERFLOW` flag is set to `TRUE`, the normal weir formula is bypassed and the overflow volume is set directly to the volume exceeding HRW. This is a simplified, computationally cheaper approximation and is not recommended when accurate spillway hydraulics are required.

LRW Penalty Cost

If the reservoir level falls below `LRW`, a cost is incurred that is proportional to both the depth of the violation and the length of the time step:

$$C_{\text{LRW}} = p \cdot \frac{\Delta t}{3600} \cdot (\text{LRW} - h)$$

where p is the penalty parameter `RES_PENALTY`, Δt is the time step in seconds, and h is the current water surface elevation [masl]. The linear dependence

on the depth of violation ensures a smooth, differentiable cost function that provides a meaningful gradient signal for optimization algorithms: a deeper violation produces a proportionally larger cost. Note that aggressive actions can impact this.

2.3 Powerstation

The PSTATION node models a hydropower power station. Unlike the reservoir node, the power station cannot store water. All water that enters the node in a given time step must leave it in the same time step via the downstream connection (DOWNLINK_IDNR).

A power station node receives water exclusively through its upstream reservoir tunnel connection. The upstream reservoir passes its water surface elevation at the start and end of the time step to the power station, which uses these values to compute the available hydraulic head (Hnetto). Figure 2.4 illustrates how a powerstation is connected to a reservoir through a tunnel.

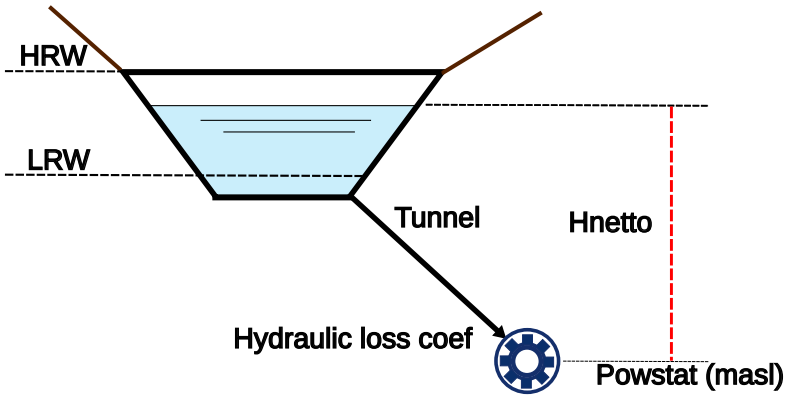


Figure 2.4: Node type powerstation.

Hydraulic Head and Power Calculation

The gross hydraulic head is computed as the average reservoir elevation over the time step, minus the turbine elevation:

$$H_{\text{brutto}} = \frac{h_{\text{start}} + h_{\text{end}}}{2} - h_{\text{turbine}}$$

where h_{start} and h_{end} are the reservoir water surface elevations [masl] at the beginning and end of the time step respectively, and h_{turbine} is the turbine centreline elevation (POWSTAT_MASL) [masl].

Head losses in the penstock are modelled using a quadratic loss formula:

$$\Delta H = k \cdot Q^2 \tag{2.6}$$

where k is the head loss coefficient (HEADLOSSCOEF) and Q is the discharge [m^3/s]. The net head is then:

$$H_{\text{netto}} = H_{\text{brutto}} - \Delta H \tag{2.7}$$

Mechanical power output for each generator is:

$$P = \eta_{\text{turbine}}(Q) \cdot \eta_{\text{generator}} \cdot \rho g \cdot H_{\text{netto}} \cdot Q \quad [\text{MW}] \tag{2.8}$$

where $\eta_{\text{turbine}}(Q)$ is the turbine efficiency at the current discharge, read from the turbine efficiency curve, and $\eta_{\text{generator}}$ is the static generator efficiency. Energy produced in the time step is:

$$E = P \cdot \frac{\Delta t}{3600} \quad [\text{MWh}]$$

Generators and Turbine Efficiency Curve

A power station may have one or more generators (`NR_GENERATORS`, maximum `MAX_NR_GENERATORS`). Each generator has its own:

- A piecewise-linear *turbine efficiency curve* (`TURBINE_CURVE`) mapping discharge [m^3/s] to efficiency [%].
- A maximum discharge (`GENERATOR_MAX_DISCHARGE`) [m^3/s].
- An action variable $a \in [0, 1]$, where the actual discharge is $Q = a \cdot Q_{\max}$.

Each generator requires a corresponding action column in the actions input file.

Uniform normalised turbine efficiency curve

As a faster alternative to the general `TURBINE_CURVE`, a generator's efficiency can be specified using a uniformly-sampled normalised curve via the `UNIFORM_NORMALIZED_CURVE` keyword. Because the sample points are equally spaced in normalised discharge, the correct table entry can be found by direct index calculation rather than a binary search, making simulation runs significantly faster when many function evaluations are performed.

Definition The curve is defined by exactly $N = 11$ efficiency values η_k sampled at normalised discharges

$$Q_n^{(k)} = \frac{k}{N-1}, \quad k = 0, 1, \ldots, N-1 \tag{2.9}$$

i.e. at $Q_n \in \{0.0, 0.1, 0.2, \ldots, 1.0\}$, where the normalised discharge is defined as

$$Q_n = \frac{Q}{Q_{\max}} \tag{2.10}$$

and $Q_{\max}$ is the `GENERATOR_MAX_DISCHARGE` for that generator.

Interpolation at runtime Given an actual discharge Q, the efficiency is evaluated by:

$$Q_n = Q \;/\; Q_{\max} \tag{2.11}$$
$$i = \lfloor Q_n \cdot (N-1) \rfloor \tag{2.12}$$
$$t = Q_n \cdot (N-1) - i \tag{2.13}$$
$$\eta = \eta_i + t\,(\eta_{i+1} - \eta_i) \tag{2.14}$$

This is a simple linear interpolation with $O(1)$ index access. The returned fractional efficiency is $\eta/100$.

Topology file syntax The keyword replaces `TURBINE_CURVE` inside a `GENERATOR` block and must always be followed immediately by `GENERATOR_MAX_DISCHARGE`:

```
GENERATOR 0
UNIFORM_NORMALIZED_CURVE 11
0.0   0.0
0.1  50.0
0.2  75.0
0.3  86.0
0.4  90.0
0.5  92.0
0.6  91.5
0.7  90.0
0.8  88.0
0.9  85.0
1.0  82.0
GENERATOR_MAX_DISCHARGE 4.0
```

The first column is the normalised discharge Q_n and is for human readability only; the parser reads only the second column (efficiency in percent). The integer after `UNIFORM_NORMALIZED_CURVE` must equal $N = 11$; any other value will cause a model error.

Constraints and notes

- `UNIFORM_NORMALIZED_CURVE` and `TURBINE_CURVE` are mutually exclusive within a single generator block.
- Exactly 11 efficiency values must be provided, corresponding to $Q_n = 0.0, 0.1, \ldots, 1.0$.
- Efficiency values are given in percent [%]; they are divided by 100 internally before use.
- If the discharge passed to the efficiency function is below 10^{-6} m^3/s the function returns zero without consulting the curve.
- Discharges above `GENERATOR_MAX_DISCHARGE` are treated as an error; the simulation will abort with a diagnostic message.

Headloss in the tunnel

The `HEADLOSSCOEF` k in eq. (2.6) which is used to estimate head loss eq. (2.7) and eq. (2.8) is derived from the Darcy Weisbach equation:

$$h_f = f_d \cdot \frac{LV^2}{2gD} \tag{2.15}$$

where f_d is the dimensionless Darcy friction factor, L is the tunnel length, V is the speed of water, g is the gravitational acceleration and D is the tunnel diameter.

Substituting $V = \frac{Q}{A}$ into eq. (2.15) and comparing the resulting expression with eq. (2.6) yields the tunnel head loss coefficient

$$k = \frac{f_d L}{2gDA^2}, \tag{2.16}$$

depending on the Darcy friction factor, tunnel length, and the tunnel diameter and cross-sectional area. The Darcy friction factor may be found in tables for different surface roughnesses.

Penstock Configuration

Two penstock configurations are supported, selected with `SHARED_PENSTOCK`:

`SHARED_PENSTOCK TRUE` All generators draw from a single common penstock. Head loss is computed from the combined total discharge of all generators, and the resulting net head applies equally to all.

`SHARED_PENSTOCK FALSE` Each generator has its own independent penstock. Head loss and net head are computed separately for each generator based on its individual discharge.

Minimum Discharge

The parameter POWSTAT_MIN_DISCHARGE defines the minimum operational discharge [m^3/s] of the power station. If the total discharge falls below this threshold (but is greater than zero), a warning is issued. This is currently implemented as a soft constraint: the simulation continues but flags the violation in the log. The minimum discharge is used primarily to indicate a technically meaningful lower operating limit for the turbines.

Local Energy Equivalent (LOCAL_ENERGY_EQUIVALENT)

The LOCAL_ENERGY_EQUIVALENT [kWh/m^3] is a power station specific factor used to convert the water volume stored above LRW in the reservoirs upstream of it into an estimate of the remaining producible energy. It is a static coefficient and should be estimated using representative values for the head and efficiency of each power station and its reservoir.

The energy equivalent is used together with RESTPRICE in the input pricefile to estimate the value of the water remaining in the river system at the end of the simulation.

Automatic Minimum Flow (AUTO_QMIN)

If AUTO_QMIN is set to a positive value, the power station enforces a minimum flow through the turbines regardless of the action variable. This is intended for situations where a regulatory minimum flow must pass through the power station itself, rather than being bypassed to a channel. If the action-based discharge falls below AUTO_QMIN, the flow is raised to meet it.

Start and Stop Costs

A start/stop cost (POWSTAT_STARTSTOP) [EUR] is incurred each time a generator transitions between the off state and the on state, or vice versa. The cost is split equally between the start event and the stop event, so each transition contributes POWSTAT_STARTSTOP/2 to the total cost. This penalty discourages frequent cycling, which is physically costly in real plant operation.

To make the cost differentiable (automatic differentiation) the state of a generator is not a hard on/off switch. A smooth indicator $s(a)$ of the action a is used: $s = 0$ for $a \leq 0.01$, $s = 1$ for $a \geq 0.03$, and a smoothstep $s = u^2(3 - 2u)$ with $u = (a - 0.01)/0.02$ in between. The cost in time step t is

$$\text{cost}_t = \frac{\texttt{POWSTAT_STARTSTOP}}{2} \left(s(a_{t-1}) - s(a_t)\right)^2 . \tag{2.17}$$

For actions below 0.01 or above 0.03 this is exactly the cost of the hard on/off rule: half the fee for every start or stop, and nothing in steady operation. An action between 0.01 and 0.03 counts as a partly running generator and gives a partial cost. In the first time step the previous state is taken from the state file: running ($s = 1$) if the initial power is above 0.01, otherwise stopped.

Aggressive Action Penalty

If the requested discharge in a time step exceeds the available water volume in the upstream reservoir, the power station discharge is forced to zero and a large penalty cost is applied. This mechanism can be useful in e.g. optimization. It denies requesting physically impossible discharges while still providing a strong gradient signal to drive the solution away from infeasible regions.

2.4 Channel

The `CHANNEL` node models the transport of water through a river reach between two nodes. It introduces a travel time delay and attenuation of the flood wave, representing the physical processes of translation and dispersion in a natural watercourse. Like the power station, the channel produces no electricity. The water volume stored in the channel is included in the remaining water in the system, and hence added as economic value.

A channel node receives water from its upstream neighbour through the standard `up_inflow` mechanism and routes it to a single downstream node specified by `DOWNSTREAM_NODE_IDNR`. If the downstream identifier is set to `-9`, the channel is the terminal node of the river system (ocean) and its outflow leaves the modeled domain.

Flow routing in HERSS can be done with one out of four different methods.

A. Cascaded Linear Reservoirs

B. Lagged Cascaded Linear Reservoirs

C. Nonlinear Natural Channel Routing

D. Routing using Manning's approach

2.4.1 Routing Model: Cascaded Linear Reservoirs

Flow routing in HERSS is based on a series of N cascaded linear reservoirs (`N_CASCADE_LINRES`). Figure 2.5 show a graphical representation of three linear reservoirs in series. Linear and non-linear reservoirs have been used extensively in hydrology in various forms. There are numerous publications dating as early as 1930s (Zoch [1934], McCarthy [1938], Nash [1957], Dooge [1959], Chow et al. [1988], Maidment [1993], Mateo Lázaro et al. [2015]). The formulation is conceptually equivalent to a Nash-cascade-type routing model Nash [1957]. Each linear reservoir has a storage constant $K = K_{\text{total}}/N$, where K_{total} is the total travel time parameter (`K_TRAVELTIME_HOURS`) distributed equally across all reservoirs. The number of cascaded reservoirs controls the shape of the unit hydrograph: a single reservoir ($N = 1$) gives maximum attenuation, while increasing N reduces attenuation and produces a response closer to a pure time delay.

Each linear reservoir is updated analytically within each time step using the exact discrete-time solution for a linear reservoir under constant inflow:

$$S_{t+\Delta t} = S_t \, e^{-\Delta t/K} + K \left(1 - e^{-\Delta t/K}\right) I_t$$

where S_t is the storage [m^3] at the start of the time step, I_t is the inflow [m^3/s], K is the reservoir time constant [s], and Δt is the time step length [s]. The average outflow over the time step is:

$$Q_{\text{out}} = \frac{S_t + I_t\,\Delta t - S_{t+\Delta t}}{\Delta t}$$

The output of each reservoir becomes the input to the next, and the output of the final reservoir is the routed channel discharge passed to the downstream node. This formulation is analytically exact for constant inflow within a time step and remains numerically stable for any time step length, which is important when HERSS is run with variable or large time steps.

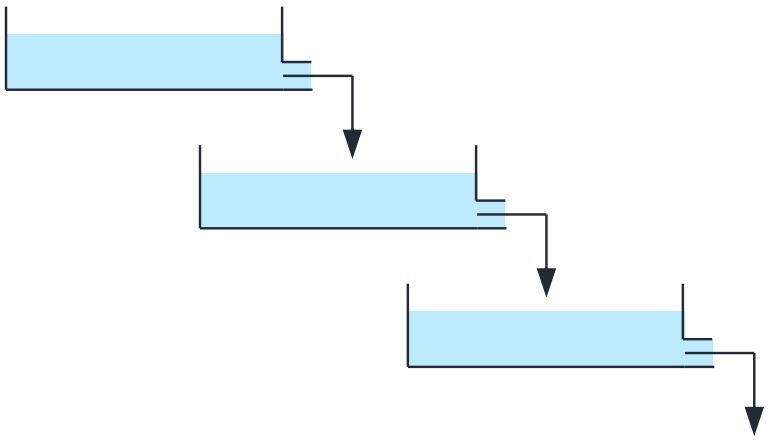


Figure 2.5: Cascaded linear reservoirs

2.4.2 Routing Model: Nonlinear Reaches

Routing method C (`N_REACHES`, `REFERENCE_FLOW`, `REFERENCE_TRAVEL_TIME`, `NONLINEAR_COEFFICIENT`) is a series of N nonlinear reservoirs (reaches). The outflow of a reach with storage S [m^3] is

$$Q_{\text{pot}} = a\,S^p,$$

where p is `NONLINEAR_COEFFICIENT` and a is chosen so that the reach releases `REFERENCE_FLOW` when the whole channel holds the volume `REFERENCE_FLOW` $\times$ `REFERENCE_TRAVEL_TIME`, shared equally between the reaches. The outflow is evaluated from the storage at the start of the time step, and the storage is updated by

$$Q_{\text{out}} = Q_{\text{pot}} \wedge \frac{S}{\Delta t}, \qquad S_{t+\Delta t} = S_t + (I_t - Q_{\text{out}})\,\Delta t,$$

where $\wedge$ is a smooth minimum, so that a reach cannot release more water than it holds. The smooth minimum (`smin_nn` in `smoothing.h`) is exactly Q_{pot} unless $S/\Delta t$ is within 0.1 % of it, so the cap only differs from a hard minimum in that narrow zone. It is differentiable, which is what the backward pass (`Channel::Backward`) needs. Unlike method A, the update is explicit: the outflow of a step depends on the storage at the start of the step, so an inflow reaches the outflow of a channel with N reaches N steps later at the earliest.

`NONLINEAR_COEFFICIENT` must be at least 1. The value 1 is a linear reservoir. For $p < 1$ the travel time would grow with the flow, which a river does not do, and the slope $dQ/dS = apS^{p-1}$ of an empty reach would be infinite. Such values are rejected when the topology file is read.

2.4.3 Parameters

`K_TRAVELTIME_HOURS` The total travel time parameter [hours], equivalent to the sum of the individual reservoir time constants $K_{\text{total}} = N \cdot K$. For a single linear reservoir this equals the mean residence time of the channel reach. Physically it can be estimated as the approximate travel time of a flood peak through the reach. The maximum allowed value is `MAX_TRAVELTIME_HOURS` (currently 240 hours, i.e. 10 days).

`N_CASCADE_LINRES` The number of cascaded linear reservoirs N. Increasing N reduces attenuation and sharpens the routed hydrograph. A value of 1 gives the most spreading; values of 3–5 are typical for natural river reaches. The maximum allowed value is `MAX_NR_CASCADED_RESERVOIRS` (currently 10).

Pure delay (routing method B). Adding `ROUTING_DELAY_HOURS` L to the node passes the outflow of the cascade through a delay of L whole hours, $Q_{\text{out}}(t) = Q_{\text{cascade}}(t - L)$. The first L hours are released from the lag queue of the state file. The delay requires a time step of one hour. The model is linear and smooth, and it has an adjoint in HERSS-AD like the plain cascade: the gradient of the value function with respect to the cascade outflow at step s is the gradient with respect to the channel outflow at step $s + L$. Water that enters in the last L hours is still in the channel when the run ends, and has no value. At present only routing methods A and B can be used with `AUTODIFF TRUE`.

For river reaches under gradually varying, subcritical flow conditions, the kinematic wave approximation provides a practical framework for estimating travel time. Under this approximation, the flood wave celerity c is related to the mean flow velocity V by

$$c = \frac{5}{3}V, \tag{2.18}$$

meaning that a flood wave travels approximately 67% faster than the water itself. The mean flow velocity can be estimated from Manning's equation,

$$V = \frac{1}{n} R^{2/3} S_0^{1/2}, \tag{2.19}$$

where n is Manning's roughness coefficient, R is the hydraulic radius [m], and S_0 is the channel bed slope [-]. The travel time through a reach of length L [m] is then

$$T = \frac{L}{c} = \frac{3nL}{5R^{2/3} S_0^{1/2}}. \tag{2.20}$$

In practice the following approaches are useful for estimating `K_TRAVELTIME_HOURS`:

- **Manning's equation** as above, using topographic maps or a digital elevation model to extract slope, and assuming a representative hydraulic radius and roughness.
- **Observed hydrographs**: if discharge measurements exist at two stations along the reach, the lag between corresponding flood peaks gives a direct estimate of the wave travel time.
- **GIS tools**: can compute reach-averaged slope and length automatically from a DEM.
- **Rule of thumb**: for steep rivers ($S_0 \sim 0.005$–0.02) travel times are typically in the range of 0.1–2 hours per km of reach. For flatter lowland rivers the travel time may be several times longer.
- **Manning's roughness coefficient**: n is typically in the range of 0.025-0.1 for natural streams, where the lower values represent large rivers with little restrictions and rough sections, and the larger numbers represent smaller streams with more rough patches and large boulders.

The number of cascaded linear reservoirs `N_CASCADE_LINRES` controls attenuation independently of the travel time. A single reservoir ($N = 1$) gives maximum spreading of the flood peak; values of $N = 3$–5 are typical for natural river reaches.

2.4.4 Minimum Flow (`QMIN`)

A minimum flow requirement can be defined for a channel node using `QMIN`. If the routed outflow Q_{out} falls below the required minimum Q_{min} in a time step, a

penalty cost is incurred, no additional actions is executed. The penalty is the fee of the period, c [Euro/hour], scaled by the unmet fraction of the requirement:

$$\text{cost} = c\,\frac{\Delta t}{3600}\,\frac{\text{relu}_\epsilon(Q_{min} - Q_{out})}{Q_{min}}, \tag{2.21}$$

where $\text{relu}_\epsilon(x)$ is a smooth version of $\max(0, x)$ that is exactly zero for $x \leq 0$, equals $x - \epsilon$ for $x \geq 2\epsilon$ and is quadratic in between. The smoothing width is $\epsilon = \min(0.01, 0.05\,Q_{min})$ [m^3/s]. A requirement that is met costs nothing. The cost grows with the unmet fraction, a half fee when half of the requirement is missing, and so on. Because of the smoothing, the full fee is not reached: when nothing is released the cost is the fraction $1 - \epsilon/Q_{min}$ of the fee, at least 95 %. A smooth penalty is used so that the cost has a gradient everywhere below the requirement, which is needed for gradient based optimisation (automatic differentiation). Up to five seasonal periods with different minimum flow requirements can be defined.

If `QMIN` is used downstream of a reservoir enforcing the `OUTLET_AUTO_QMIN`, note that the number of cascaded linear reservoirs `N_CASCADE_LINRES`, the time resolution Δt in the simulation, and the `K_TRAVELTIME_HOURS` all affect the outflow from this (`CHANNEL`) node. As the outflow exhibits asymptotic behaviour and for a finite time never approaches the inflow value, the outflow can lie slightly below the requirement. With the smooth penalty this costs almost nothing: a shortfall of 10^{-3} [m^3/s] costs 2.5×10^{-5} of the fee for a requirement of 1 [m^3/s].

When `QMIN` is used, HERSS expect the following arguments:

```
QMIN <nr_periods>
   <start date DD.MM> <end date DD.MM> <Qmin m3/s> <penalty Euro>
...
```

where `nr_periods` is the integer describing the number of defined periods listed below (Maximum number of 5). The same date verification procedure is used as for `OUTLET_AUTO_QMIN`.

2.4.5 State File

The channel state file entry stores the water volume currently held in each of the N cascaded linear reservoirs at the end of the simulation:

```
NODE CHANNEL <idnr> <nodename> <S_0_Mm3> <S_1_Mm3> ... <S_{N-1}_Mm3>
```

One storage value is written per cascaded reservoir, in Mm^3. These values are read back at the start of the next simulation run as initial conditions for the routing model, enabling correct continuity across chained simulations.

2.4.6 Channel Start and End State Files

The channel records in the start-state and end-state files use the same whitespace-delimited ASCII format. The start-state file is read during initialization, while the end-state file is written with the channel state at the end of the simulation. Each channel record has the form

```
NODE CHANNEL <idnr> <name> <routing-method> N <count> <
    ↪ state-values...>
```

The routing method is configured by the channel topology. The method name in the state file identifies the format of the stored values and is checked against the topology configuration. It does not select the routing method. Do not include the literal word `METHOD` in the current format.

For `A_CASCADEDLINRES`, the values are the storage volumes in the individual linear reservoirs. The value count must equal `N_CASCADE_LINRES` in the topology.

```
NODE CHANNEL 2 VANAROSEN A_CASCADEDLINRES N 3 0.00100
    ↪ 0.00200 0.00300
```

For `B_LAGGED_CASCADEDLINRES`, the first values are the storage volumes in the linear reservoirs. They are followed by a lag queue, whose count must match `ROUTING_DELAY_HOURS`. The first lag value is the volume released at the next time step.

```
NODE CHANNEL 2 VANAROSEN B_LAGGED_CASCADEDLINRES N 3
    ↪ 0.00100 0.00200 0.00300 LAG 4 0.01000 0.02000
    ↪ 0.03000 0.04000
```

The arrow indicates display wrapping only. Each channel record must be written as one physical line in the ASCII state file; the arrow is not part of the record.

For `C_NONLINEARRESCASCADE`, the values are the stored volumes in the channel reaches. The count is the number of reaches configured by `N_REACHES`.

```
NODE CHANNEL 2 VANAROSEN C_NONLINEARRESCASCADE N 3 0.01000
    ↪  0.02000 0.03000
```

For `D_MANNING`, the values are also the stored volumes in the channel reaches, with the count set by `N_REACHES`.

```
NODE CHANNEL 2 VANAROSEN D_MANNING N 3 0.01000 0.02000
    ↪ 0.03000
```

All stored volumes are expressed in million cubic metres (`Mm3`). Values appear in reach or reservoir order. The end-state writer formats these values to five digits after the decimal point.

For the two cascaded-reservoir methods, the reservoir-storage count must match `N_CASCADE_LINRES`. For the lagged method, a lag-queue count that does not match

ROUTING_DELAY_HOURS is initialized to zero. For the nonlinear and Manning methods, a reach-storage count that does not match N_REACHES causes the channel to use its configured initialization instead. State rows containing only untagged values, without the routing method and count, are not in the current format.

2.5 Pump

The PUMP node models a PSH facility, which is mutch like a power staion node but in reverse. It lifts water from a source RESERVOIR to a target RESERVOIR while consuming electricity. Its source and target reservoirs are given via PUMP_SOURCE_IDNR and PUMP_TARGET_IDNR rather than the usual downstream connection. Only one pump unit per node is supported, unlike power stations, where multiple generators are supported. Water below PUMP_INLET_MASL in the source reservoir cannot be withdrawn.

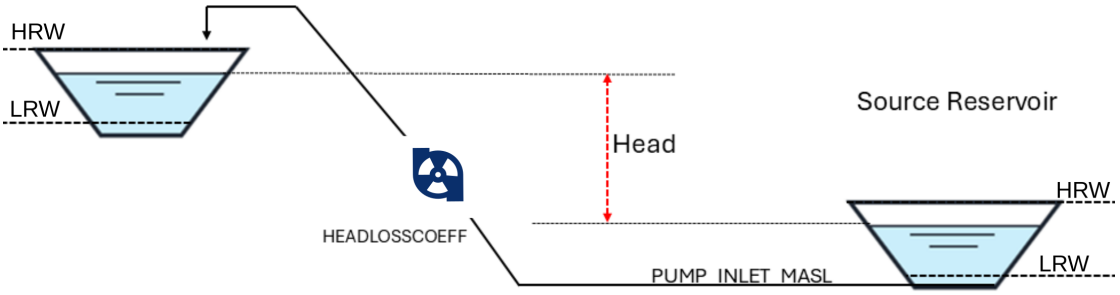


Figure 2.6: Node type PUMP. Illustration of the connection of the pump between its source and target reservoirs.

In the simulation of each time step, while the movement of water flow from the most upstream node to the most downstream node (IDNR 0 to IDNR $(N-1)$), the pump moves water from its source to the target reservoir when the simulation reaches the pumps IDNR. When all movement of water is simulated within each time step, the power and cost of electricity are calculated based on average head between target and source reservoirs, before the next time step is simulated.

2.5.1 Hydraulic Head and Power Calculation

The gross hydraulic head is computed as the average value between the target and source reservoirs within a time step, here as target minus source:

$$H_{\text{brutto}} = \left(\frac{h^{\text{tgt}}_{\text{start}} + h^{\text{tgt}}_{\text{end}}}{2} \right) - \left(\frac{h^{\text{src}}_{\text{start}} + h^{\text{src}}_{\text{end}}}{2} \right) \tag{2.22}$$

where h are the reservoir water surface elevations denoted $_{start}$ and $_{end}$ for the beginning and end of a time step respectively, and ^{tgt} and ^{src} for target and source reservoir, respectively.

Head losses are added to the head the pump must overcome, unlike Eq. 2.7 for power stations. This is done using the same quadratic formula as for poser stations and give net head as:

$$H_{\text{netto}} = H_{\text{brutto}} + k \cdot Q^2 \tag{2.23}$$

where k is the head loss coefficient (`HEADLOSSCOEF`) for the pump and Q is the pumped water [m^3/s].

Electrical power drawn from the grid is:

$$P = \frac{\rho g \cdot H_{\text{netto}} \cdot Q}{\eta_{\text{pump}}(Q) \cdot \eta_{\text{motor}}} \quad [\text{MW}] \tag{2.24}$$

where $\eta_{\text{pump}}(Q)$ is read from a `PUMP_CURVE/UNIFORM_NORMALIZED_CURVE` (same format as the turbine efficiency curve, Section **??**) and η_{motor} is `STATIC_MOTOR_EFFICIENCY`.

2.5.2 Topology File Syntax

```
NODE PUMP <idnr> <nodename>
PUMP_SOURCE_IDNR <idnr>
PUMP_TARGET_IDNR <idnr>
PUMP_INLET_MASL <masl>
MAX_DISCHARGE <m3/s>
HEADLOSSCOEF <k>
STATIC_MOTOR_EFFICIENCY <0-1>
PUMP_STARTSTOP <value>
UNIFORM_NORMALIZED_CURVE 11
0.0     <0-100>
...
1.0     <0-100>
ENDNODE
```

2.5.3 Start and Stop Costs

A start/stop cost (`PUMP_STARTSTOP`) [EUR] is incurred each time the pump transitions between the off state ($a = 0$) and the on state ($a > 0$), or vice versa, exactly as for a power station (Section **??**). The cost is split equally between the start event and the stop event, so each transition contributes `PUMP_STARTSTOP`/2 to the total cost.

2.5.4 Numbering Requirement

Both `PUMP_SOURCE_IDNR` and `PUMP_TARGET_IDNR` must be higher than the pump's own `IDNR` since the pumped water is moved before either reservoir each time step.

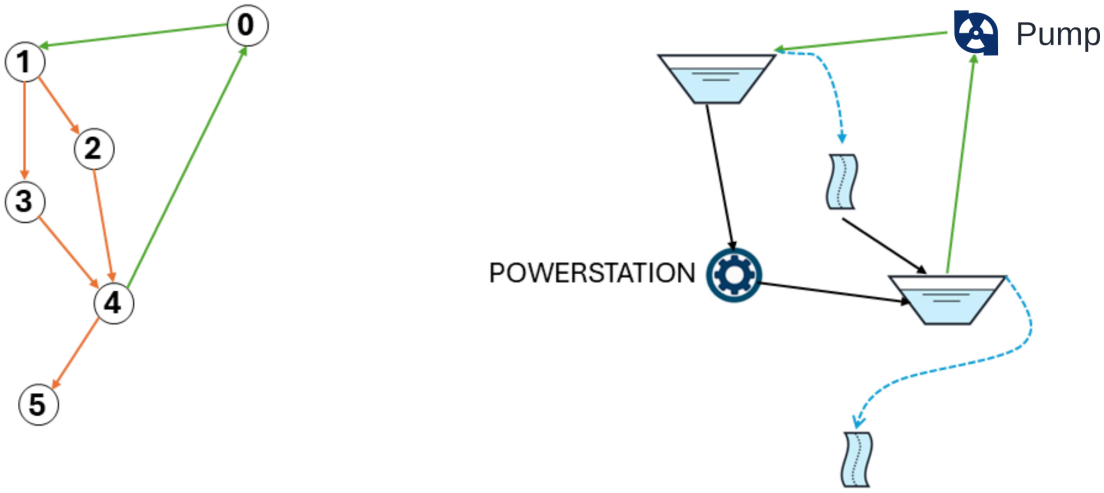


Figure 2.7: Schematic of the `Mini_pump_test` system, and how the nodes are connected in the topology. `IDNR` 0 is the pump, and `IDNR` 3 is the power station. This illustrates how a pump can move water without following the normal rule of water flow from a node with a lower to a higher `IDNR`, but may move water between tow nodes with `IDNR` higher than itself.

2.6 Intake

The INTAKE node models how water from a creek/stream is taken into an inlet tunnel. Typically the intake is a small dam or pond in the river. The intake is designed so it will capture all flow levels below a specified capture capacity [m^3/s].

in the same time step via the downstream connection (DOWNLINK_IDNR).

An intake node receives water through its upstream incoming natural streamflow. The intake will collect all the inflowing water up to a threshold.

connection. The upstream reservoir passes its water surface elevation at the start and end of the time step to the power station, which uses these values to compute the available hydraulic head (Hnetto). Figure 2.8 illustrates how an intake is connected.

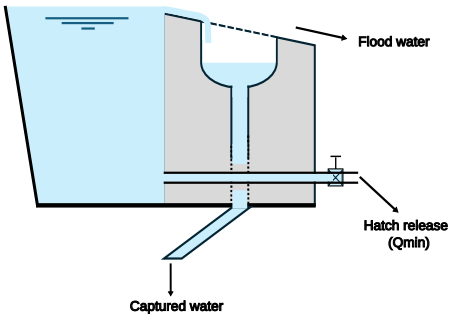


Figure 2.8: Schematic of node type Intake

The parameters are:

Keyword	Type	Description
FLOODWATER_IDNR	int	Id number to downstream node receiving flood water
CAPTURE_CAPACITY	float	Capture capacity of intake [m^3/s]
CAPCAP_IDNR	int	Id number to downstream node receiving captured water
HATCH_RELEASE	float	Amount of water released [m^3/s]
HATCHREL_IDNR	int	Id number of downstream node receiving
VOLUME	float	Total volume of intake reservoir [Mm^3]

2.7 Automatic Differentiation (AD)

HERSS includes a manually implemented adjoint, or reverse-mode derivative, for calculating sensitivities of the simulation value function. This is not a general-purpose automatic-differentiation package: the forward simulation remains the normal HERSS simulation, while reverse equations are implemented for supported node and routing operations. The adjoint uses double-precision values.

2.7.1 Value function and gradients

The differentiated objective is the HERSS value function

$$\mathcal{V} = \sum_{n=1}^{N_{\text{nodes}}} \sum_{t=0}^{T-1} (\text{income}_{n,t} - \text{cost}_{n,t}) + \text{restprice}\, E_{\text{remaining}}, \tag{2.25}$$

where $E_{\text{remaining}}$ is the energy value assigned to active water remaining in the river system at the end of the simulation. The remaining energy is calculated using the local energy equivalents of the downstream power stations (or, for systems without power stations, the reservoir energy equivalents).

A gradient named `x_grad` denotes

$$\texttt{x_grad} = \frac{\partial \mathcal{V}}{\partial x}. \tag{2.26}$$

For example, `action_grad` is the change in value function per unit change in an action. Gradients are sensitivities of the value function, not new simulated state variables.

2.7.2 How the adjoint is evaluated

The adjoint is evaluated after the forward simulation and value-function calculation. HERSS first clears previous gradients, then seeds the reverse pass with the derivatives of the objective: each timestep's node profit has a seed of one, and the terminal reservoir states receive additional seeds from the value assigned to remaining active water.

The backward pass then traverses timesteps in reverse order and nodes in reverse topological order. Each node propagates the gradients it receives to its upstream inputs, state variables, and actions. This includes propagation through time-dependent reservoir and channel states. The reverse ordering is important: downstream contributions must be accumulated before the corresponding upstream node is processed.

The equations include smooth versions of selected limits and thresholds so that useful derivatives are available near operating constraints. As a result, gradients near these transitions describe the implemented smoothed model; they need not be identical to derivatives of an otherwise equivalent model using hard, discontinuous limits.

2.7.3 Enabling gradient output

Set the following keyword in the global configuration file:

```
AUTODIFF TRUE
```

The default is `FALSE`. With AD enabled, HERSS performs its normal simulation, calculates the value function, runs the backward pass, and writes a gradient output file under `OUTPUTDIR`, named `gradients_<SYSTEMNAME>.txt`.

The output is grouped by node and includes available gradients such as `profit_grad`, `Power_grad`, `res_Mm3_grad`, `res_masl_grad`, `Hbrutto_grad`, and `up_inflow_grad`. Power-station action sensitivities are reported per generator; reservoir output includes the sensitivity to the initial reservoir volume. The file is intended for inspection. Its current layout is not a regular CSV table, so scripts should not assume fixed columns or one row per timestep without first checking the output.

2.7.4 Optimizing actions with Adam

HERSS can use the adjoint gradients to optimize actions with the Adam optimizer. Set `ADAM_OPTIMIZER TRUE` in the global configuration. The optimizer maximizes the value function by minimizing its negative. By default, it optimizes generator actions; the selectable control groups are `GENERATORS`, `HATCH`, and `ALL`. Generator and hatch actions are constrained to the interval $[0, 1]$.

For example:

```
ADAM_OPTIMIZER TRUE
OPTIMIZE_CONTROLS GENERATORS
LEARNING_RATE 0.02
MAX_ITERATIONS 1000
PATIENCE 100
START_ACTIONS FILE
OPTIMIZED_ACTIONFILE actions_optimized.txt
```

The iteration limit, learning rate, and other optimizer settings can be adjusted in the global file. The learning rate decays during the run, and the action bounds are enforced after each update. Adam stops at the iteration limit or when a configured stopping condition is met. It writes the optimization history and optimized

actions under `OUTPUTDIR`. HERSS then runs its normal output sequence using the optimized actions.

2.7.5 Example datasets

The `ad_mini_utahps` dataset is a compact daily example with one reservoir, one power station, and a routed channel. Its global file enables gradient output but leaves Adam optimization commented out. It is suitable for inspecting the gradients produced by a forward run.

The `ad_utahps_hourly` dataset demonstrates the adjoint and optimization on a larger hourly system. Its global file enables both `AUTODIFF` and `ADAM_OPTIMIZER`; it optimizes the generator actions, writes an Adam history file and an optimized action file, and then runs HERSS with those actions. The example contains four power stations and 8,760 hourly steps.

2.7.6 Current limitations

The adjoint is implemented only for supported model operations; enabling `AUTODIFF` does not make every HERSS feature differentiable. In particular, the reservoir backward pass requires reservoir geometry and a spillway, and does not support reservoir curves, fast overflow, overflow curves, or connected pumps. Pump and intake backward methods are not implemented. Channel backward propagation currently supports routing methods A (cascaded linear reservoirs), B (lagged cascaded linear reservoirs), and C (nonlinear reservoir cascade), subject to the channel configuration being supported by the code. Check the topology and node features before using gradients for optimization.

The adjoint tests compare supported gradients with finite-difference estimates on selected cases. For a new topology or feature combination, validate its gradients against finite differences before relying on them for optimization.

2.8 Input

This section describes in more detail the input files required to run HERSS. To some extent it is a repetition of what was said in earlier chapters. Feel free to skip it or have quick look ☺. An efficient way to learn about the input files and their format is to just open the testdata and use it as a starting point in your own project.

2.8.1 The Global Configuration File

The global configuration file is the single entry point to HERSS. It is passed as a command-line argument when the model is executed: `herss.exe global.txt`. The file collects all top-level settings in one place: the system name, paths to input and output directories, names of all other input and output files, and a small number of run-time flags. Lines beginning with `#` are treated as comments and ignored.

Table 2.1 describes each keyword.

Table 2.1: Keywords in the global configuration file.

Keyword	Description
SYSTEMNAME	Name of riversystem. Used in output filenames.
INPUTDIR	Path to the directory containing all input files.
TOPOLOGYFILE	Name of the topology file
INFLOWFILE	Name of the inflow time series file.
PRICEFILE	Name of the electricity price file. Also defines the simulation period and time step.
ACTIONFILE	Name of the file containing release and production actions.
STARTSTATEFILE	Name of the state file defining initial conditions (reservoir levels, channel storage).
OUTPUTDIR	Path to the directory where all output files are written.
OUTSTATEFILE	Name of the output state file written at the end of the simulation.
WRITE_NODEFILES	Write individual node output files (1 = yes, 0 = no).
PRINT_GLOBAL_INFO	Print global system information to screen (TRUE/FALSE).
PRINT_ECONOMIC_INFO	Print economic summary to screen (TRUE/FALSE).

2.8.2 The Topology File

The topology file defines the physical structure of the river system: which nodes exist, what type they are, how they are connected, and all their physical parameters. It is the largest and most detailed of the input files.

File Structure

The file consists of a sequence of node blocks. Each block opens with a `NODE` declaration and closes with `ENDNODE`. Lines beginning with `#` are comments. The general structure of a node block is:

```
NODE  <NODETYPE>  <IDNR>  <NAME>
<keyword>  <value>
...
ENDNODE
```

where `NODETYPE` is one of `RESERVOIR`, `PSTATION`, `PUMP`, or `CHANNEL`, `IDNR` is the unique integer node identifier, and `NAME` is a descriptive label used in output files and log messages.

As described in Section 1.4, node identifiers must be assigned in strict upstream-to-downstream order. HERSS simulates nodes in ascending order of `IDNR` at each time step, so all upstream nodes must have lower identifiers than the nodes they drain into.

Downstream connections are specified differently for each node type:

- `RESERVOIR`: downstream connections are implicit in the outlet keywords (`OUTLET_TUNNEL`, `OVERFLOW_CURVE`, `OUTLET_HATCH`, `OUTLET_AUTO_QMIN`), each of which carries a downstream `IDNR`.
- `PSTATION`: the downstream node is specified with `DOWNLINK_IDNR`.
- `CHANNEL`: the downstream node `IDNR` is given on the `NODE` line itself. A value of `-9` indicates no downstream connection, i.e. the system outlet.
- `PUMP`: This node only specifies which nodes water is moved between with two nodes of higher `IDNR` than itself by specifying `PUMP_SOURCE_IDNR` and `PUMP_TARGET_IDNR`.

The parameters within each node block are described in detail in section **??**.

The topology file is used to specify all the nodes that the riversystem consists of. Each node needs an identification number (idnr). The first node needs the idnr to

be zero (0), and the following nodes increasing. Upstream and downstream nodes also needs to be specified. The lowest outlet node (ocean or most downstream part of riversystem is identifies if the downstream node is set to -9). The topology file is typically made by looking at maps/GIS description of your system.

2.8.3 The Price file

The price file supplies the electricity spot price time series and the end-of-horizon water value used to evaluate remaining storage. It is the file that also defines the simulation period and the number of time steps: HERSS counts the data rows in the price file during initialisation to determine the total number of time steps (`stps`). All other time series files (inflow, actions) must contain the same number of rows.

File Format

The file has a fixed two-line header followed by one data row per time step:

```
RESTPRICE   <value>
Date        Price
<timestamp> <price>
<timestamp> <price>
...
```

`RESTPRICE` The water value, or rest price, at the end of the planning horizon [currency/MWh]. This value is used to estimate the economic worth of water remaining in the reservoirs at the end of the simulation. It must appear on the first non-comment line.

`Date Price` A fixed column header line. Must appear as the second non-comment line.

Timestamp Each data row begins with a timestamp in one of the formats supported by HERSS: `yyyymmddhh`, `yyyymmdd`, `yyyy-mm-dd`, or `yyyy-mm-dd-hh`. The time step Δt is derived automatically from the difference between consecutive timestamps, enabling variable temporal resolution within the same file (see also section 2.10).

Price The electricity spot price for the time step [currency/MWh]. HERSS is currency-agnostic; any consistent currency unit may be used. The test dataset uses EUR/MWh.

It is assumed that the entire river system lies within a single price area, so one price series applies uniformly to all power stations and penalty calculations.

2.8.4 The Inflow file

The inflow file provides the lateral inflow time series for each reservoir in the river system. Inflow represents natural runoff entering the river network at a specific location, for example precipitation-driven runoff from a catchment draining directly into a reservoir. Inflows are given in [m^3/s] and are provided for each simulation time step.

File Format

The file has a single header line followed by one data row per time step. The header identifies the columns by their reservoir node `IDNR`:

```
Date_NodeID  <res_idnr_0>  <res_idnr_1>  ...
<timestamp>  <Q_0>     <Q_1>     ...
<timestamp>  <Q_0>     <Q_1>     ...
...
```

The keyword `Date_NodeID` is mandatory and must appear as the first token on the header line. After the keyword, the remaining tokens on the same line are the integer node identifiers (`IDNR`) for the reservoirs in the system. The first column of each data row is a timestamp in one of the formats accepted by HERSS (`yyyymmddhh`, `yyyymmdd`, `yyyy-mm-dd`, or `yyyy-mm-dd-hh`). The timestamps must match exactly the timestamps in the price file. HERSS validates each row and will report an error on any date mismatch.

Each of the remaining column gives the inflow [m^3/s] entering the node whose `IDNR` was listed in the corresponding header position. The number of data rows must equal the number of time steps defined by the price file.

For a larger system with inflow entering multiple reservoir nodes, the header and data rows would contain multiple columns, for example:

```
Date_NodeID  0     3     7
2026050700   1.50  0.80  2.10
2026050701   1.35  0.75  1.95
...
```

Here nodes 0, 3, and 7 each receive their own inflow series. All other nodes in the system receive zero lateral inflow by default.

2.8.5 The Actions File

The actions file specifies how each controllable element in the river system is operated at every time step. An action is a dimensionless control signal with values in the range [0.0, 1.0], where 0.0 means the element is fully closed or off and 1.0 means it operates at its maximum capacity. Intermediate values represent partial operation.

Actions are defined for the following controllable elements:

- **Power station generators**: the action sets the discharge through the turbine as a fraction of the generator's maximum discharge - (`GENERATOR_MAX_DISCHARGE`). Each generator in a multi-generator station has its own action column.
- **Reservoir hatches** (`OUTLET_HATCH`): the action controls the gate opening, scaling the discharge linearly between the defined minimum and maximum hatch flows.
- **Pumps**: the action sets the discharge through the pump as a fraction of the pumps maximum discharge (`MAX_DISCHARGE`).

File Format

The format mirrors the inflow file: a single header line followed by one data row per time step.

```
Date_NodeID  <node_gen>  <node_gen>  ...
<timestamp>  <action>    <action>    ...
<timestamp>  <action>    <action>    ...
...
```

`Date_NodeID` Mandatory keyword on the header line. The remaining tokens on the header line identify each column using the notation `<idnr>_<generator_index>`, where `idnr` is the node identifier and `generator_index` is the zero-based generator number within that node. For example, `5_0` refers to generator 0 of the power station at node 5, and `5_1` refers to its second generator.

Timestamp First column of each data row. Must use one of the HERSS date formats (`yyyymmddhh`, `yyyymmdd`, `yyyy-mm-dd`, or `yyyy-mm-dd-hh`) and must match the timestamps in the price file exactly. HERSS validates each row and reports an error on any mismatch.

Action values Each remaining column gives the action $a \in [0.0, 1.0]$ for the element identified in the header. The number of data rows must equal the number of time steps defined by the price file.

Example

In the mini uTAHPS example, only node 1 (`SVOLETJONN`) has one generator, so the actions file has a single data column:

```
Date_NodeID  1_0
2026050700   0.66
2026050701   0.00
2026050702   0.00
2026050703   0.00
2026050704   1.00
...
```

For a system with multiple power stations and generators, the header lists all controllable elements and the data rows supply each action independently:

```
Date_NodeID  1     5_0   5_1
2026050700   0.00  0.80  0.70
2026050701   0.70  0.75  0.00
...
```

Here node 1 is a reservoir with the hatch outlet in operation (no underscore). Node 5 is a powerstation with two generators operating independently. Only elements that have a corresponding action column are controlled; all other nodes receive no action signal and behave according to their passive or automatic settings.

2.8.6 The State File

The state file defines the initial hydraulic condition of the river system at the start of the simulation. It is the mechanism that connects successive simulation runs: the output state file written at the end of one run (`OUTSTATEFILE`) can be used directly as the input state file (`STARTSTATEFILE`) for the next run, enabling seamless chaining over a long planning horizon.

File Format

The file contains one line per node that carries state. Lines beginning with `#` are comments and are ignored. Each node line follows the pattern:

```
NODE  <NODETYPE>  <IDNR>  <NAME>  <value(s)>
```

Different node types each have their own format:

`NODE RESERVOIR` *idnr name fr* The reservoir initial condition is given as a *filling fraction* $f_r \in [0.0, 1.0]$, where 0.0 corresponds to the Lowest Regulated Water level (LRW) and 1.0 corresponds to the Highest Regulated Water level (HRW). Values outside this range are physically possible (the reservoir may be below LRW or spilling above HRW) and are accepted by HERSS. The initial stored volume is computed as:

$$V_{\text{start}} = V_{\text{LRW}} + f_r \cdot (V_{\text{HRW}} - V_{\text{LRW}})$$

`NODE PSTATION` *idnr name P* The power station initial condition is the production [MWh] at the last time step of the previous run. This value is used to evaluate start/stop cost transitions at the first time step of the new simulation. A value of `0.0` means the station was off at the end of the previous period.

`NODE CHANNEL` *idnr name* S_0 S_1 $\ldots S_{N-1}$ The channel initial condition is the water volume stored in each of the N cascaded linear reservoirs [Mm^3]. The number of values must exactly match `N_CASCADE_LINRES` as defined for this node in the topology file. These values initialise the routing model so that the channel carries the correct amount of in-transit water from the start.

`NODE PUMP` *idnr name P* The PHS facility's initial condition is the energy consumed [MWh] at the last time step of the previous run, written as a positive value. This value is used to evaluate start/stop cost transitions at the first time step of the new simulation. A value of `0.0` means the pump was off at the end of the previous period.

Example

The mini uTAHPS state file *start_state.txt*:

```
# STATEFILE MINI UTAHPS
```

```
# NODE RESERVOIR IDNR NAME RES_FR
NODE RESERVOIR 0 HJELLE 0.67

# NODE PSTATION IDNR NAME MWh
NODE PSTATION 1 SVOLETJONN 0.0

# NODE CHANNEL IDNR NAME LINRES1_Mm3 LINRES2_Mm3 LINRES3_Mm3
NODE CHANNEL 2 VANAROSEN 0.001 0.002 0.003
```

Here the `HJELLE` reservoir starts at 67 % of its active storage range, the power station `SVOLETJONN` was off at the end of the previous period, and the `VANAROSEN` channel carries three small water volumes corresponding to its three cascaded linear reservoirs.

2.9 Output Files

After a simulation has finished, HERSS writes a set of plain-text output files to the directory specified by `OUTPUTDIR` in the global configuration file. For a system named `mini_uTAHPS_hourly`, the output folder will contain the following files:

- *node0_HJELLE.txt*, *node1_SVOLETJONN.txt*, *node2_VANAROSEN.txt* — individual node output files (written only if `WRITE_NODEFILES 1`)
- *reservoirs_mini_uTAHPS_hourly_out.txt* — reservoir filling fractions for all reservoirs
- *riversystem_mini_uTAHPS_hourly_output.txt* — aggregated river system summary
- *outstate.txt* — end state of the simulation

The exact filenames depend on the system name (`SYSTEMNAME`) and the output state filename (`OUTSTATEFILE`) set in the global configuration file.

2.9.1 Node Output Files

When `WRITE_NODEFILES` is set to `1`, HERSS writes one output file per node, named *nodeidnr_NODENAME.txt*. These files contain the most detailed simulation results and are the primary source for time series plots.

Each file begins with a short header identifying the node type and name, followed by a time series with one row per time step. The columns depend on the

node type and are described in Section **??**. Typical variables include inflow, reservoir water level, reservoir filling fraction, tunnel and hatch discharge, overflow, power production, hydraulic head, income, and cost.

2.9.2 Reservoir Filling Fractions File

The file *reservoirs_systemname_out.txt* contains the time series of filling fractions for every reservoir in the system, with one column per reservoir. An example from the mini uTAHPS system:

```
Riversystem mini_uTAHPS_hourly reservoir fractions [fr]
YYYY MM DD HH HJELLE
2026 5 7 0 0.6695
2026 5 7 1 0.6701
...
```

This file is particularly convenient for plotting reservoir levels and checking storage trends across the simulation period.

2.9.3 River System Summary File

The file *riversystem_systemname_output.txt* provides a compact summary of the entire simulation. It contains:

- A node inventory listing each node, its type, and its remaining water volume [Mm^3] at the end of the simulation.
- A global water balance: starting volume, total inflow, total outflow, and ending volume, with a check that the balance closes to zero.
- Aggregated economic results, including total production [MWh], total income, total costs (LRW penalty, start/stop costs, minimum flow violations), total profit, the estimated value of remaining water, and the final value function.

The value function is the quantity typically used by optimisation algorithms to evaluate the quality of an action sequence:

$$V = \text{tot_profit_Euro} + \text{tot_remaining_Euro}$$

Output State File

At the end of every simulation, HERSS writes an output state file (named by `OUTSTATEFILE` in the global configuration). The format is identical to the input state file described above. For reservoirs, the written value is the simulated filling fraction f_r at the final time step. For power stations it is the final production value. For channels it is the volume held in each cascaded linear reservoir at the end of the last time step. This file can immediately be used as `STARTSTATEFILE` for a subsequent run.

2.10 Multi-Temporal Resolution and date formats

Multi-temporal resolution is introduced in order to reduce computational burden in long simulation horizons while retaining sufficient process realism. When the full simulation period is represented at a uniformly fine timestep, such as hourly resolution, the number of timesteps becomes large and model runtime increases substantially. This is especially important in applications involving repeated simulations. This could be optimization, scenario analysis, or stochastic sampling.

Table 2.2: Example of variable temporal resolution input. The time step Δt is derived internally from the difference between consecutive timestamps.

Datetime	**Price**	**Δt** [s]	**Resolution**
2024010100	45.2	3 600	hourly
2024010101	43.8	3 600	hourly
2024010102	41.1	3 600	hourly
2024010103	39.5	3 600	hourly
...	...	...	...
2024010200	52.3	86 400	daily
2024010300	55.1	86 400	daily
2024010400	50.7	86 400	daily
...	...	...	...
2024010800	48.0	604 800	weekly
2024011500	46.5	604 800	weekly
2024012200	44.2	604 800	weekly

In addition, the required temporal detail is not constant across all parts of the simulation horizon. Short-term operational processes may require fine temporal resolution, whereas long-term planning, seasonal storage dynamics, and economic

evaluation may be represented adequately at daily or weekly resolution. A variable-resolution framework therefore allows computational effort to be allocated where it is most informative.

The use of multiple temporal resolutions is also justified by the fact that input data often have different effective information content over time. High-frequency forcing may be important during rapid hydrological events or short-term market fluctuations, while lower-frequency representation may be sufficient during slowly varying periods. Accordingly, a multi-temporal routing formulation provides a practical compromise between computational efficiency and process representation.

The main methodological requirement is that the routing model must remain mass-conserving and numerically consistent when the timestep changes. This is necessary to ensure that coarser temporal resolution reduces computational cost without introducing unacceptable distortion of storage dynamics, attenuation, or delay. Table 2.2 shows how variable timestep length can be used in the input files. In the last timestep we assume Δt is the same as in the preceding step.

Chapter 3

Tutorial and feature guide

This chapter provides a tutorial and presentation of several test datasets designed to explore various functionality in the HERSS model.

3.1 Testdata Reservoir Cascade A

We will now explore the test dataset provided in the folder: `./res_casc_A/`. The node structure and the topology of this dataset is illustrated in Figure 3.1. The system has two reservoirs, one powerstation and three channels. There are two features that we will explore in this example: hatch release directly from a reservoir, and the use of multi generators at a powerstation. We also use daily simulation steps.

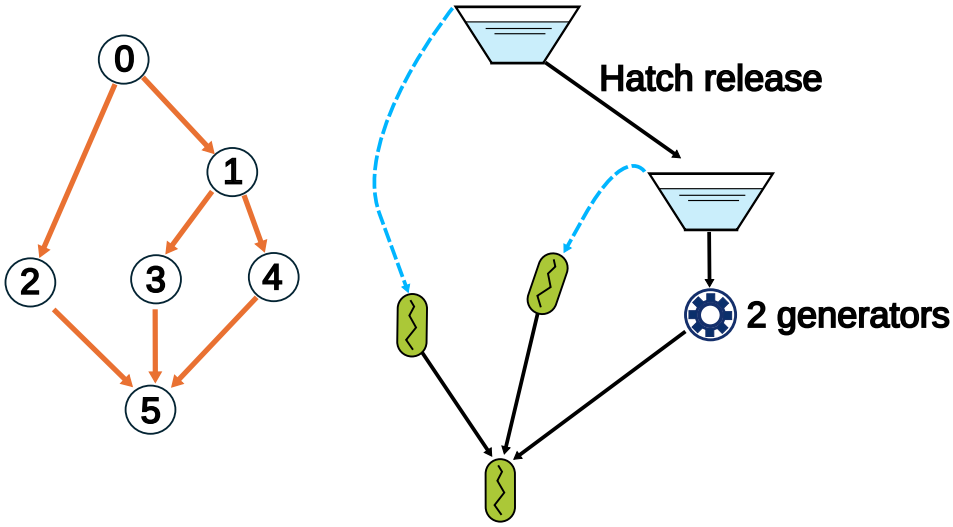


Figure 3.1: Node structure and topology of the reservoir cascade test dataset

Now let's open the *actions.txt* inputfile. You should see the following:

```
Date_NodeID 0 4_0 4_1
20220901 0.7 0.8 0.8
20220902 0.8 0.8 0.0
20220903 0.9 0.8 0.0
...
```

In the first row of the actions file, you can see four columns. The first column is the date, and in the next three we have actions. The action for this system consist of the hatch release from reservoir RES_A, which is in node 0. This corresponds to column two. The last two columns are the action for generator 0 and 1, in the powerstation named PSTAT_B, in node 4. Note the name convention.

In the topology file, at line 29 and 30, you will see the flags setting usage of hatch outflow from reservoir RES_A (node 0). We can see that the maximum outflow from the hatch is set to 6.0 m^3/s.

```
# OUTLET_HATCH downstream_nodeid, qmin_hatch, qmax_hatch, hatch_masl
OUTLET_HATCH 1 0.0 6.0 848.0
```

Run herss for this system with the command `../../herss.exe global.txt`.

Inside the folder `./res_casc_A/output/` you will see all the different output files generated by HERSS. Now let's take a look at the simulation results for reservoir RES_A (node 0). Figure 3.2 shows the inflow to the reservoir in (a), the hatch outflow and overflow in (b), and the reservoir filling in percent in (c). The maximum hatch outflow was set to 6.0 m^3/s in the topology file. This means that actions with values of 1.0 will release maximum hatch flow (6.0 m^3/s). We can see that in Figure 3.2 (b), the hatch outflow is never above this level. We can also see that the reservoir is above full in two time periods. Overflow is then activated and we see that overflow is larger than zero.

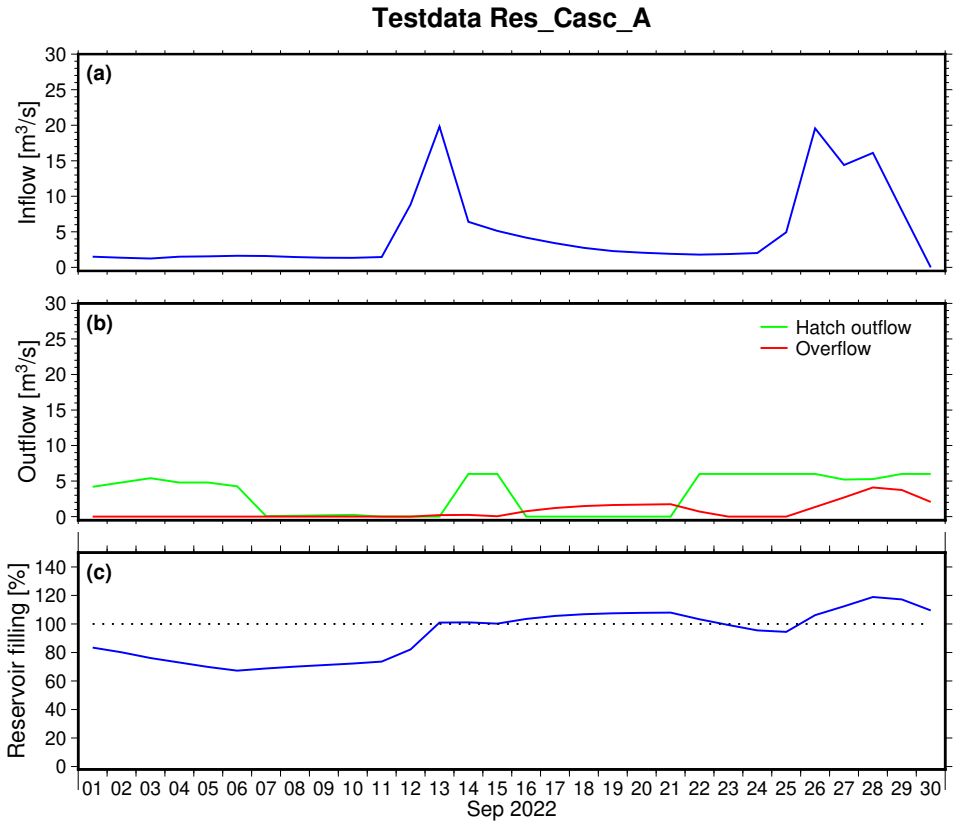


Figure 3.2: Simulation results for the reservoir RES_A (node 0)

Figure 3.3 shows the simulation results for the powerstation PSTAT_B (node 4), and the connected reservoir, RES_B. The figure has four subplots. In subplot (a), we see the inflow to the upstream reservoir. In (b) the reservoir filling is plotted. The actions for generator 0 and 1 are illustrated in subplot (C). In the lowest subplot in Figure 3.3, the total power is shown. In the beginning of the time period only one generator is running. This proudces around 40 MWh pr day. Later in the time period, around 22-26 Sep, both generators are running at fairly high actions. The system then produces around 75 MWh pr day. This illustrates the usage of more than one generator in a single powerstation.

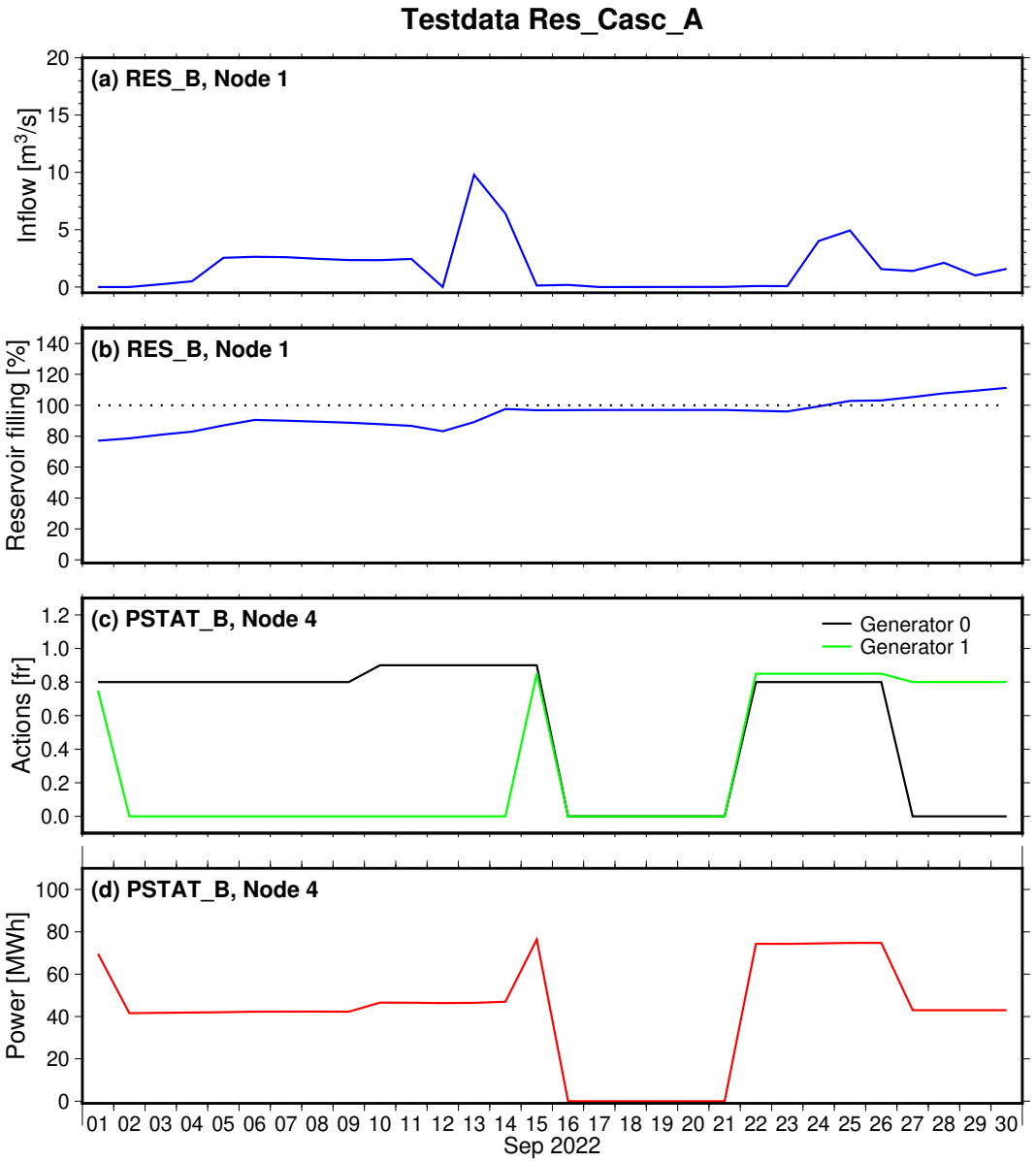


Figure 3.3: Simulation results for the powerstation PSTAT_B (node 4), and the connected reservoir RES_B (node 4)

3.2 Testdata UTAHPS Intake

We will now explore the test dataset provided in the folder: `./utahps_intake/`. The node structure and the topology of this dataset is illustrated in Figure 3.4.

The system has xxxx

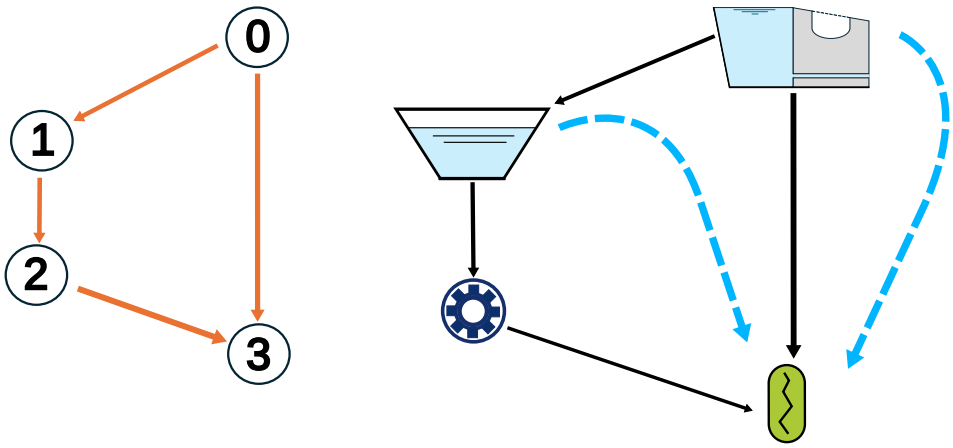


Figure 3.4: Intake nodestructure

3.3 Running HERSS from Python

3.3.1 What is cppyy?

`cppyy` is an automatic Python-C++ bindings library that allows Python code to call C++ classes, functions, and variables directly at runtime, without writing any wrapper code. Under the hood, `cppyy` runs a small C++ interpreter that reads your header files and figures out what classes, functions, and variables exist in your C++ code. It then automatically makes those available as Python objects — so you can call a C++ function from Python just as if it were a regular Python function, without ever writing any glue code yourself.

The key advantage of `cppyy` is that no binding code needs to be written or compiled. You load a shared library and include the corresponding header file, and all classes and functions declared in that header become immediately available as Python objects.

Install `cppyy` with:

```
pip install cppyy
```

This is where the HERSS shared library file (*herss.so*) comes in handy. It is part of the HERSS source code and is made ready during compilation of HERSS.

3.3.2 Loading HERSS into Python

The two lines below are all that is needed to make the entire HERSS C++ API available in Python:

```
import cppyy
cppyy.load_library("../src/herss.so")
cppyy.include("../src/herss.h")
```

After these calls, all classes declared in `herss.h` — including `GlobalConfig`, `Dataset`, `Herss`, `Logger`, and others — are accessible through the `cppyy.gbl` namespace.

3.3.3 Initialising the Logger

HERSS uses an internal logging system. When running from Python, the `main()` function is never called, so the logger must be initialised manually before any other HERSS code is executed:

```
version      = cppyy.gbl.VERSION
version_date = cppyy.gbl.VERSION_DATE
logfilename  = f"herss_{version}_{version_date}.log"

if not cppyy.gbl.Logger.instance().init(logfilename):
print(f"Could not open log file: {logfilename}")
```

All subsequent `LOG_WARN`, `LOG_ERR`, and `LOG_MSG` calls inside C++ will write to this file.

3.3.4 Setting Up a Simulation

A HERSS simulation is set up in the same sequence as the C++ `main()` function. The following steps must be performed in order.

1. Create and configure GlobalConfig.

```
gc = cppyy.gbl.GlobalConfig()
gc.globalfile = "../data/mini_utahps_daily/global.txt"
gc.readGlobalFile()
gc.inputdir  = "../data/mini_utahps_daily/"
gc.outputdir = "../data/mini_utahps_daily/output/"
gc.SetDirectoriesAndFilenames()
gc.Diagnose()
gc.checkNrSteps()
```

2. Read input data.

```
data = cppyy.gbl.Dataset(gc)
data.readAllData()
```

3. Create the Herss object and prepare the simulation.

```
herss = cppyy.gbl.Herss(gc)
herss.prepaireSimulation(data)
herss.rs.DiagnoseRiversystemConfiguration()
```

4. Run the simulation.

```
herss.Simulate()
vf = herss.rs.CalcVF(data.restprice)
print("Value function =", vf)
```

3.3.5 Reading and Writing Simulation State

After `Simulate()` has been called, results can be read back through the `Herss` interface methods. Table 3.1 lists the most commonly used functions.

After modifying prices, inflows, or actions, call `herss.Simulate()` again. It is not necessary to call `prepaireSimulation()` again; `Simulate()` reinitialises reservoir states internally on every call.

Method	Description
`GetPrice(t)`	Electricity price at timestep t
`GetInflowInNode(t, node)`	Local inflow at node at timestep t [m^3/s]
`GetReservoir_Init_fr(idnr)`	Initial reservoir filling fraction
`GetReservoirLevel_fr(node, t)`	Reservoir filling fraction at end of timestep t
`GetAction(node, gen, t)`	Action for generator `gen` at node at timestep t
`SetPrice(t, price, restprice)`	Override price and rest price at timestep t
`SetInflowInNode(t, node, q)`	Override inflow [m^3/s] at node at timestep t
`SetReservoir_Init_fr(node, fr)`	Override initial reservoir filling fraction
`SetAction(node, gen, t, value)`	Set action for generator `gen` at timestep t

Table 3.1: Commonly used `Herss` interface methods available from Python.

3.3.6 The pyherss.py Example Script

A complete working example is provided in `py_src/pyherss.py`. This script demonstrates the full workflow described above, including reading results, overriding inputs, and re-running the simulation.

New users are encouraged to work through the script one block at a time in an interactive Python session (e.g. using Jupyter or IPython):

1. Run only the `cppyy` load and include lines. Verify that no import errors occur.
2. Create `gc` and call `readGlobalFile()`. Print `gc.nr_nodes` and `gc.stps` to confirm the configuration was read correctly.
3. Create `data` and call `readAllData()`. Print `data.price[0]` to verify that price data is loaded.
4. Create `herss`, call `prepaireSimulation(data)`, then `Simulate()`. Print the value function.
5. Experiment with `SetPrice()`, `SetInflowInNode()`, and `SetAction()`, then call `Simulate()` again and observe how the value function changes.

This incremental approach makes it easy to isolate the source of any error, since each step depends only on the steps before it.

> *Note:* `cppyy` translates C++ errors and `LOG_ERR` calls into Python exceptions. If a call terminates unexpectedly, check both the Python traceback and the HERSS log file for details.

Chapter 4

Datasets

This chapter present test datasets that are part of the HERSS repository.

4.1 Upper Tovdalen Artificial Hydro Power System (uTAHPS)

4.1.1 Riversystem

In this open source project, we have defined an artificial hydropower system that doesn't exist in the real world. We have synthetically placed a hydropower system in the upper part of the Tovdalen river system, located in southern Norway. We define this system as the Upper Tovdalen Artificial Hydro Power System (uTAHPS). The system consists of four reservoirs and four powerstations with varying nr of generators and penstock configuration. A schematic map of the system is illustrated in Figure 4.1. Red dots are powerstations, the blue regions are illustrations of the synthetic reservoirs. The drainage area for each reservoir is also noted in the figure.

Figure 4.2 shows the uTAHPS riversystem with node numbers and names, and their connections. This node-network is defined in the topology files that the HERSS model reads. Note that the uTAHPS can be modelled with fewer nodes depending on the usecase. In chapter 1 we used the uppermost three nodes of this system as an example.

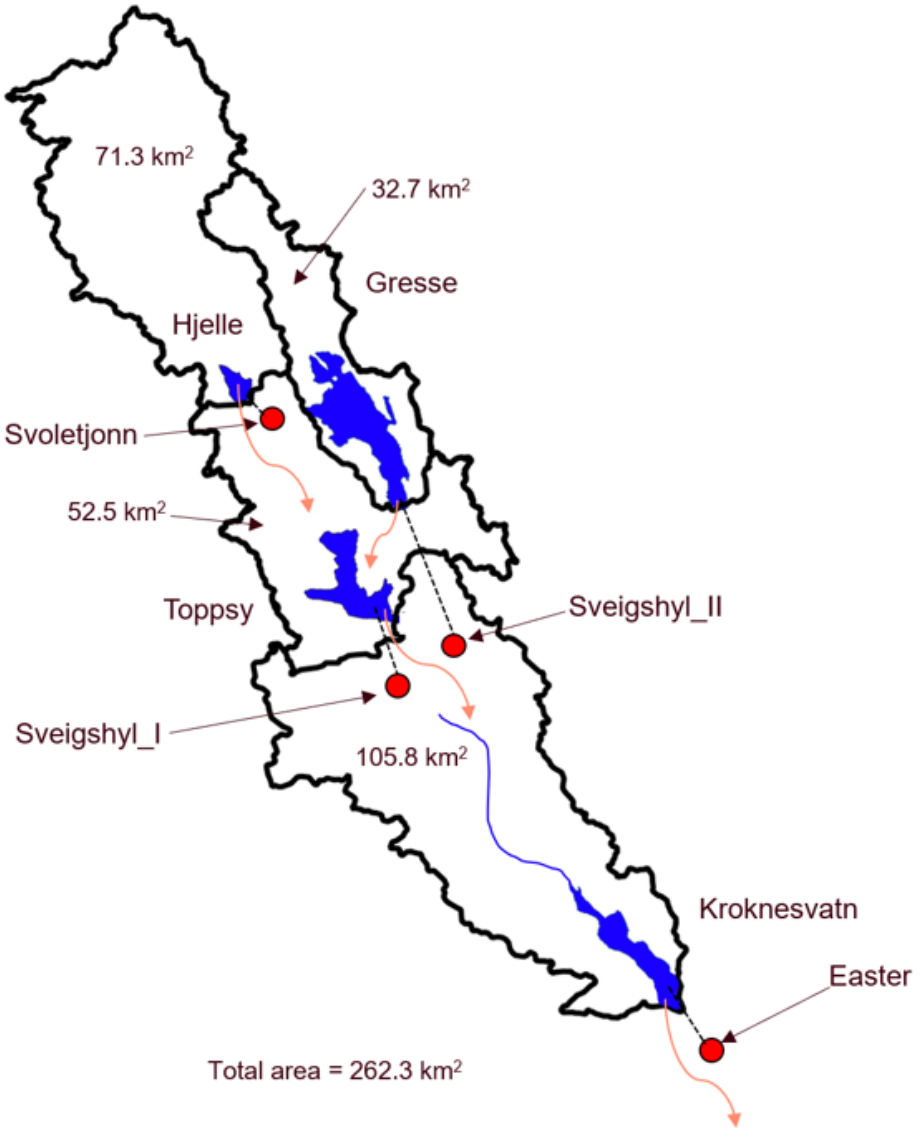


Figure 4.1: Map of Upper Tovdalen Artificial Hydro Power System

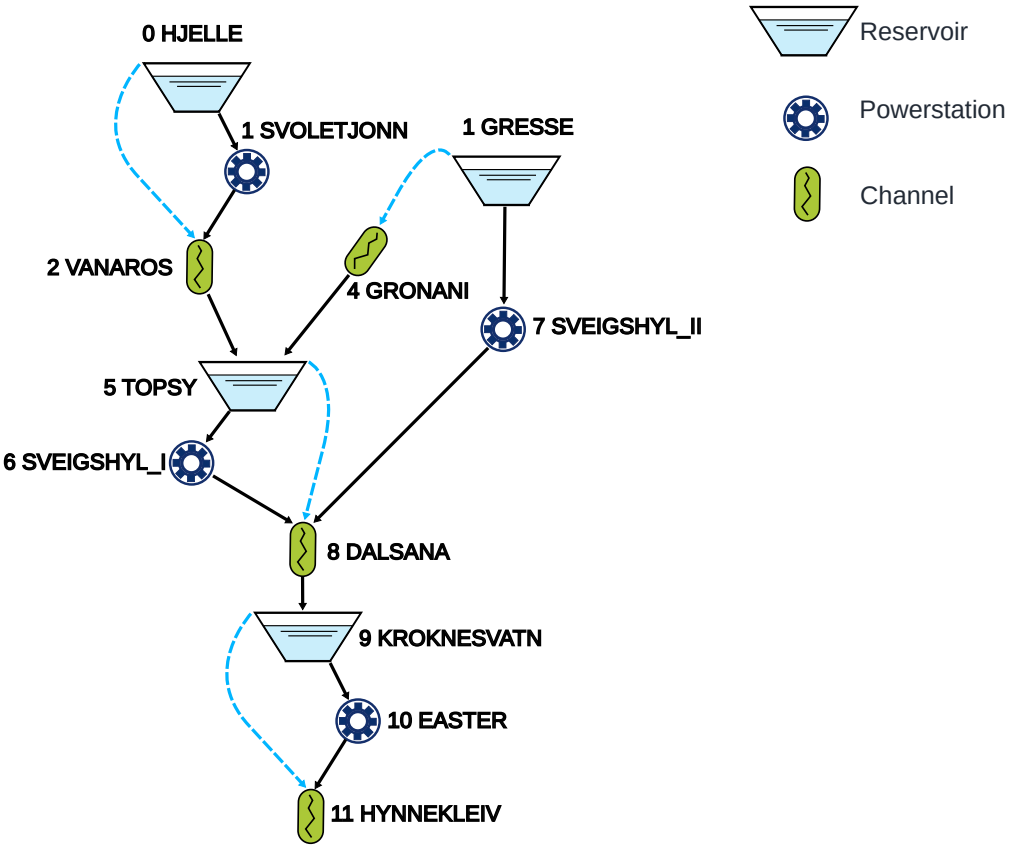


Figure 4.2: Topology of the uTAHPS. Node number and names included.

4.1.2 Timeseries

In addition to riversystem description for the uTAHPS a set of timeseries are also provided. Hourly, daily, and a multi-temporal dataset of inflow and price are provided. The selected time period goes from 1 Sep,2022 until 31 Aug, 2023.

Inflow

In the uTAHPS system it is assumed that the four reservoirs receive local inflow. The timeseries for these reservoirs are generated from observed streamflow data extracted from the Sildre open database available from The Norwegian Water Resources and Energy Directorate (www.nve.no, https://sildre.nve.no/).

The inflow time series for the four reservoirs (HJELLE, GRESSE, TOPSY, and KROKNESVATN) were derived from observed streamflow recorded at two nearby hydrological gauging stations, Austenå and Songedalsåi. Local inflow to each reservoir was estimated by scaling the observed streamflow by catchment area, with manually adjusted weights applied to account for differences in catchment characteristics. This approach is a pragmatic method when direct observations at the reservoir inlets are not available. Figure 4.3 shows the timeseries of inflow to each of the four reservoirs in uTAHPS.

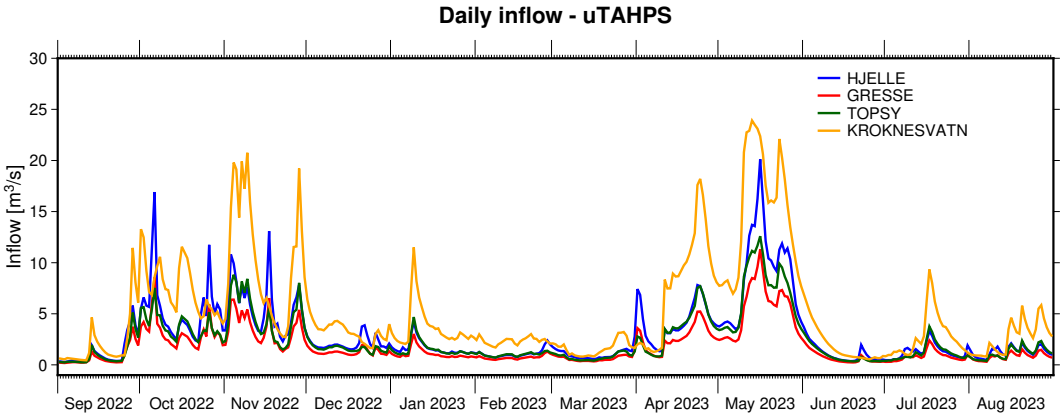


Figure 4.3: Timeseries plot of the inflow to four reservoirs in uTAHPS

Energy price

The uTAHPS system is located in the NO2 price region in Norway. Historical data for this region is available through Forbrukerrådet (https://www.forbrukerradet.no/). These data were converted to Euro/MWh assuming 1 Euro = 10 NOK and Value Added Tax of 25 %. The data can be downloaded from:

https://www.forbrukerradet.no/strompris/spotpriser/. Note that HERSS is agnostic to the currency units used. It could be NOK/MWh, Euro/MWH.

Actions

The timeseries of actions (production) for each power station were manually set in each timestep.

uTAHPS datasets

There are several different datasets available for the uTAHPS. These are located under the folder `./data/`. They have all been tested with the latest version of the ***`herss.exe`***. Some of the datasets have also been used in previous chapters. The different datasets are designed to test different types of features. Take a look in the topology files and timeseries files to see how different features are used.

```
herss/
└── data/
    ├── mini_utahps_daily/
    ├── mini_utahps_hourly/
    ├── mini_utahps_spillway/
    ├── mini_utahps_new_inputformat/
    ├── res_casc_A/
    ├── res_casc_B/
    ├── res_casc_C/
    ├── res_casc_D/
    ├── utahps_daily/
    ├── utahps_hourly/
    ├── utahps_multires/
    └── utahps_daily_new_format/
```

Figure 4.4 shows a plot of the timeseries data avaiable in the `./utahps_multires/`.

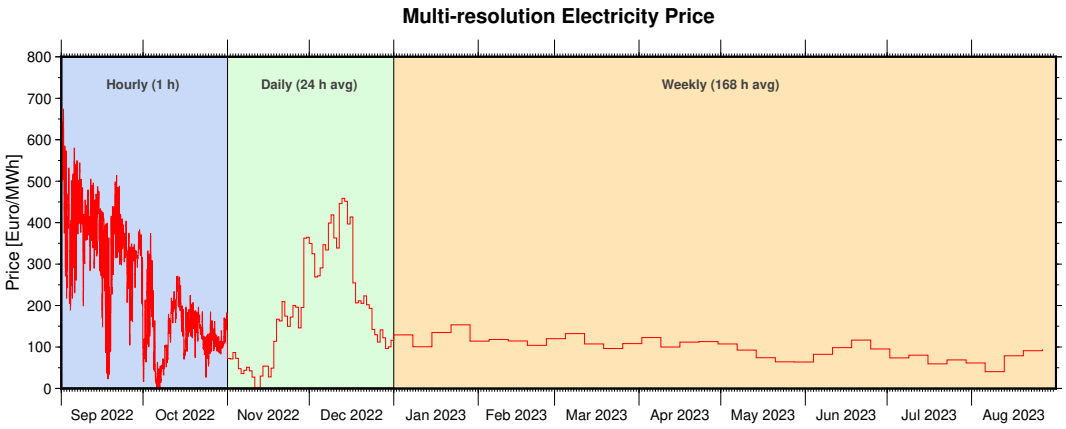


Figure 4.4: Multi-temporal timeseries plot of electricity price

Chapter 5

Topology Editor

The HERSS Topology Editor is a graphical application for creating, inspecting and editing river-system topology files. It displays reservoirs, power stations, channels, intakes and pumps as a connected network, making it easier to understand and modify a dataset.

The editor is a separate Python application. It does not perform simulations: HERSS remains the simulation engine. In addition to topology editing, the application can display results from completed HERSS runs. The editor is under active development.

5.1 Installing the Python Environment

The editor requires Python 3.11 or newer. The requirements file, `freeze_topoed_requirements.txt`, records the package versions used in the prepared environment. Installing from this file avoids automatically selecting the latest versions of the dependencies.

The following commands are for Linux. From the HERSS repository root, enter the editor directory, create a dedicated virtual environment, and install the recorded requirements:

```
cd herss-topology-editor
python3 -m venv .venv
.venv/bin/python -m pip install -r freeze_topoed_requirements.txt
```

An internet connection and Git are required. The requirements file includes an editable Git installation of the editor at a specific commit, together with pinned dependencies such as PySide6, NumPy and pyqtgraph. It also includes testing tools.

Although the editor supports Python 3.11 or newer, the selected Python version must also be supported by the pinned dependency versions. For closest re-

producibility, use the same Python version as the environment from which the requirements file was generated.

Check the installed packages for dependency conflicts with:

```
.venv/bin/python -m pip check
```

Using the local editor source instead. The requirements installation uses the Git revision recorded in the file, not necessarily the source in the current working directory. To use the local editor source with the installed dependencies, run the following additional command from the editor directory:

```
.venv/bin/python -m pip install --no-deps -e .
```

This replaces the editor installation with an editable installation of the local source. It does not reinstall dependencies; the local source must therefore remain compatible with the installed versions.

5.2 Starting the Editor

Start the application and optionally provide a topology filename:

```
.venv/bin/topoed /path/to/topology.txt
```

To start with no file loaded, use:

```
.venv/bin/topoed
```

Then select **File ¿ Open** to load an existing topology file.

5.3 Viewing and Editing a Topology

The **Topology** tab contains the graphical canvas and the **Nodes**, **Properties** and **Issues** panels. Selecting a node on the canvas or in the node list displays its parameters.

A typical editing workflow is:

1. Open an existing topology, or select **File ¿ New** and use **Edit ¿ Add node** to create a system.

2. Drag nodes to arrange the diagram. Drag empty canvas space to pan, and use **Ctrl + mouse wheel** to zoom. **View ¿ Fit to window** displays the complete network. **View ¿ Auto layout** arranges the nodes automatically.

3. Edit node parameters in the **Properties** panel. Dedicated forms are available for reservoirs, power stations, channels and intakes. Pump parameters and fields not covered by the forms can be edited in the **Raw text** tab.

4. Connect nodes by dragging from the dot on a node's right edge to another node. For a reservoir, choose the appropriate outlet when prompted. Connections can also be changed in the Properties panel. Replace generated placeholder parameters with appropriate values before running HERSS.

5. Review the **Issues** panel, then use **File ¿ Save** or **File ¿ Save As**.

Validation runs automatically when the topology is loaded or edited. It reports problems including duplicate node IDs, references to missing nodes and invalid pump ordering. HERSS simulates nodes in ascending ID order; a pump must have a lower ID than both its source and target reservoirs. The editor does not silently renumber nodes. Editor validation does not replace checking the input with HERSS.

Moving a node changes only its graphical position, not its ID or simulation parameters. Layout information is saved separately as `<topology filename>.layout.json`. The topology remains a HERSS text file, with comments and unrecognised fields preserved.

Figures 5.1 and 5.2 show example datasets displayed in the editor.

5.4 Displaying Simulation Results

Run HERSS separately, then select the **Configurator** tab and specify the run's **Results folder**. The editor reads node CSV files with an optional `manifest.json`, and supports legacy node text files as a fallback. Enable node output in the HERSS global file using `WRITE_NODEFILES 1`.

The editor attempts to locate results automatically. An explicit folder can be selected using **Browse**. The status message reports missing results, topology mismatches and results older than the topology file.

In the Configurator, define plot titles, variables and nodes. A series can select one node or all nodes of a given type. Selected series may be combined using a mean or, for suitable additive quantities, a sum.

The **Plot** tab displays three panels containing stacks of plots with a shared time axis. An editable system-summary preset provides a starting point. Plot settings are saved using **File ¿ Save** in `<topology filename>.plots.json`, separately from the topology. The Configurator and Plot tabs do not modify the topology.

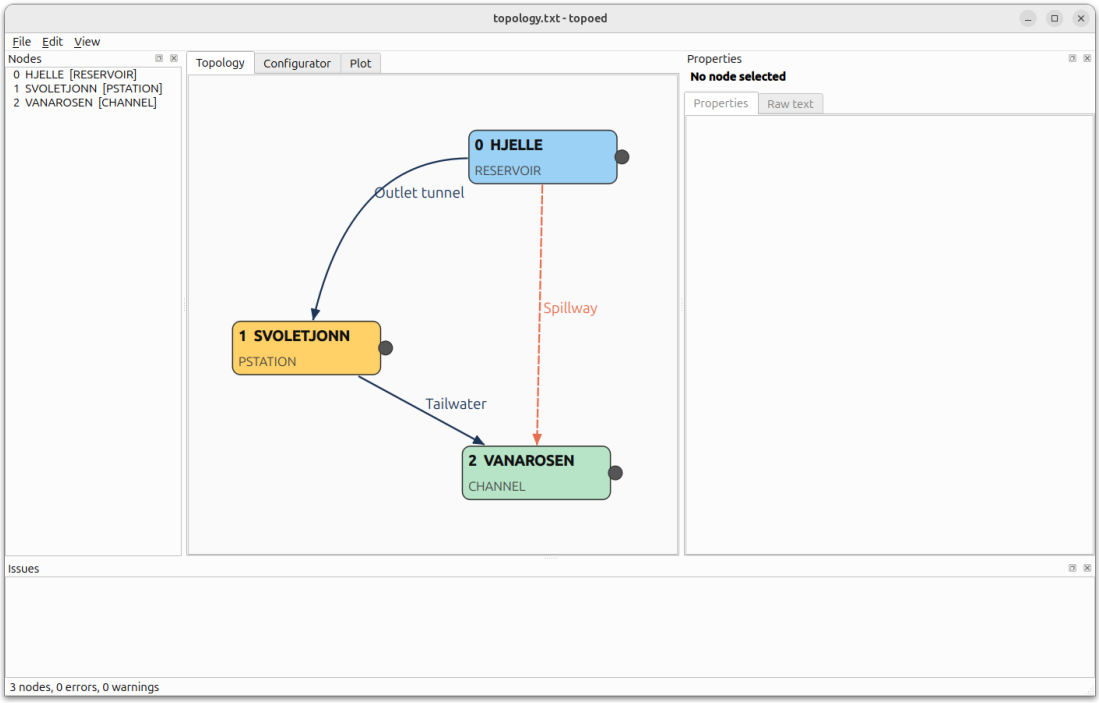


Figure 5.1: An example HERSS dataset displayed in the Topology Editor. Nodes represent system components, and links show their connections.

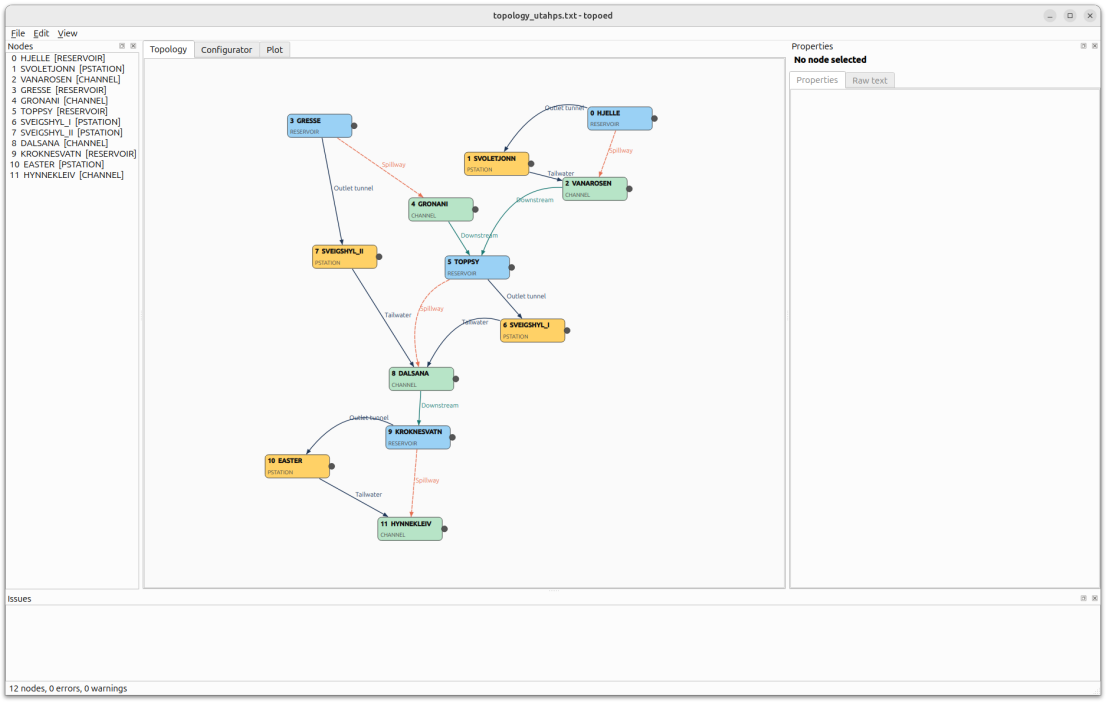


Figure 5.2: A second HERSS dataset shown in the Topology Editor. The diagram can be rearranged without changing simulation parameters or node IDs.

Chapter 6

Known Issues and Future Work

This chapter summarizes known limitations/bugs in the current version of HERSS, as well as planned improvements and ideas for future development. Some features have been implemented, but due to substantial code changes, they have not been quality controlled in the new version.

6.1 Known Issues

6.1.1 Automatic Minimum Flow Release

The automatic minimum flow release feature for reservoirs (`OUTLET_AUTO_QMIN`) is somewhat implemented, but we need to make a small testdataset showing that it works as intended.

6.1.2 Minimum Flow Constraints in Channels (QMIN)

Minimum discharge requirements in channel nodes (`QMIN`) are partially implemented. The penalty cost structure exists in the code but the time-period parsing and quality control have not been completed.

6.1.3 Maximum Adjustment Penalty

The penalty for excessive production adjustments within a day (`MAX_ADJUST`) has been implemented but has not been quality controlled.

6.1.4 Generator Max Discharge

In the current version there are are two variables with almost the same name. These are GENERATOR_MAX_DISCHARGE and POWSTAT_MAX_DISCHARGE.

Do a quality control of this, and maybe remove one of them.

6.1.5 Start of Multi-Generator

Start state and end state when using multi-generators is not implemented. All generators are set to same state.

6.2 Future Work

6.2.1 Multi-Reservoir Water Value Calculation

Extending the water value calculation to multi-reservoir systems would be useful. One approach is to use a gradient-based sensitivity method, perturbing the reservoirs individually and measuring the change in the value function.

6.2.2 Flood Level Penalty

A flood level penalty (`FLOODLEVEL_PENALTY`) has been introduced in the topology file format and the corresponding variable is read, but the penalty is not yet applied in the simulation. This need quality control.

6.2.3 Interface to python

HERSS is designed to be called from Python via `cppyy`, enabling integration with other frameworks. For now the python usage is linked to the textfile inputs required by C++. It would be nice to be able initiate a new HERSS object entirely within Python.

6.2.4 Variable Timestep Support Throughout

Variable timestep support has been introduced in the simulation loop and the water balance checks, but some parts of the codebase (notably the start/stop cost calculation and the adjustment cost calculation) still assume a fixed hourly timestep. A full audit and update of all timestep-dependent calculations is needed.

6.2.5 Parallel Simulation

For use cases with multi-scenarios (optimization), running many simulations in parallel would significantly reduce computation time. Since each `Herss` object is independent, parallelization at the scenario level is straightforward and could be implemented either in C++ or from Python.

6.2.6 Improved Topology File Validation

At present, topology file errors are caught at runtime, often deep inside a simulation. A dedicated validation pass before the simulation starts— checking that all node IDs are consistent, all downstream connections are valid, and all required keywords are present—would make error messages clearer and easier to act on. This has been partly implemented, but should be further improved.

6.2.7 Output and Visualisation

The current output is written to plain text node files and a river system summary file. Supporting structured output formats such as NetCDF or CSV with consistent column naming would simplify post-processing and integration with tools such as GMT plotting scripts.

6.2.8 Automatic generation of node networks

Putting together the topology file is time consuming and complicated for larger riversystems. We need to design a method for automatic generation of topology files based on map data.

References

V.T. Chow, D. R. Maidment, and L. W. Mays. *Applied Hydrology, Chapter 7*. McGraw-Hill, 1 edition, 1988.

J. C. I. Dooge. A general theory of the unit hydrograph. *Journal of Geophysical Research*, 64(2):241–256, 1959.

D.R. Maidment. *Handbook of hydrology*. McGraw-Hill, 1 edition, 1993.

Jesús Mateo Lázaro, José Ángel Sánchez Navarro, Alejandro García Gil, and Vanesa Edo Romero. A new adaptation of linear reservoir models in parallel sets to assess actual hydrological events. *Journal of Hydrology*, 524: 507–521, 2015. ISSN 0022-1694. doi: https://doi.org/10.1016/j.jhydrol.2015.03.009. URL https://www.sciencedirect.com/science/article/pii/S0022169415001808.

G.T. McCarthy. *The unit hydrograph and flood routing*. Conference of North Atlantic Division, U.S. Army Corps of Engineers, 1938.

J. E. Nash. The form of the instantaneous unit hydrograph. *International Association of Scientific Hydrology Publication*, 45:114–121, 1957.

R. T. Zoch. On the relation between rainfall and streamflow. *Monthly Weather Review*, 62(9):315–322, 1934.